

SOLIDWORKS API (.NET VBA C#) — Examples (27/50):

Get_DimXpertManager_Info_Example_VB

Get_Faces_Affected_by_Draft_Feature_Exam

SOLIDWORKS API Documentation (converted from CHM)

Get DimXpertManager Info Example (VBA)

This example shows the usage of several interfaces that provide access to information on the DimXpertManager tab of the Management Panel in SOLIDWORKS Design.

The IDimXpertManager interface exposes schema information on the DimXpertManager tab.

This macro demonstrates several ways of acquiring IDimXpertManager on an open document:

- ISelectionManager.GetSelectedObject6()
- IModelDocExtension.DimXpertManager
- IConfiguration.DimXpertManager

This macro also demonstrates how to get the features and annotations displayed on the DimXpertManager tab using:

- ISelectionManager.GetSelectedObject6()
- IDimXpertManager.DimXpertPart

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\cover_with_geometric_tolerances.sldprt.
' 2. Select the DimXpertManager tab in the Management Panel.
' 3. Open the Immediate window.
' 4. Add a reference to SOLIDWORKS version DimXpert type library.
'
' Postconditions: Inspect the Immediate Window.
'
' NOTE: Because this part is used elsewhere, do not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As IModelDoc2
Dim swModelDocExt As IModelDocExtension
Dim swSelMgr As ISelectionMgr
Dim swConfig As IConfiguration
Dim swFeature As IFeature
Dim swAnn As IFeature
Dim swSchema As IDimXpertManager
Dim swDXPart As IDimXpertPart
Dim msgStr As String
Dim msgStr2 As String
Dim displaytype As swDimXpertTreeDisplay_e
Dim boolstatus As Boolean
Dim n As Long
```

Sub main()

```
Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swModelDocExt = swModel.Extension
Set swSelMgr = swModel.SelectionManager

' Select a DimXpert schema using swSelectType_e.swSelSWIFTSHEMA type
boolstatus = swModelDocExt.SelectByID2("Scheme15", "SWIFTSHEMA", 0, 0, 0, False, 0, Nothing, 0)
' Get IDimXpertManager (swSchema) for the DimXpert schema using ISelectionManager.GetSelectedObject6()
Set swSchema = swSelMgr.GetSelectedObject6(1, -1)
Debug.Print (swSchema.SchemaName) + " acquired through ISelectionManager"

' Get IDimXpertManager (swSchema) for a DimXpert schema using IModelDocExtension.DimXpertManager()
Set swSchema = swModelDocExt.DimXpertManager("Default", True)
Debug.Print (swSchema.SchemaName) + " acquired through IModelDocExtension"

' Get IDimXpertManager (swSchema) for a DimXpert schema using IConfiguration.DimXpertManager()
Set swConfig = swModel.GetConfigurationByName("Default")
Set swSchema = swConfig.DimXpertManager(True)
Debug.Print (swSchema.SchemaName) + " acquired through IConfiguration"

msgStr = (swSchema.SchemaName) + " shows a "
displaytype = swSchema.TreeDisplay
Debug.Print ""

If (displaytype = swDimXpertTreeDisplay_Flat) Then
    Debug.Print msgStr + "flat display of information on DimXpert tab"

ElseIf (displaytype = swDimXpertTreeDisplay_Annotation) Then
    Debug.Print msgStr + "annotation-based display of information on DimXpert tab"

ElseIf (displaytype = swDimXpertTreeDisplay_Feature) Then
    Debug.Print msgStr + "feature-based display of information on DimXpert tab"

End If

' Select a DimXpert annotation using swSelectType_e.swSelSWIFTANNOTATIONS type
boolstatus = swModelDocExt.SelectByID2("Flatness1", "SWIFTANN", 0, 0, 0, False, 0, Nothing, 0)
' All DimXpert features and annotations returned by GetSelectedObject6 are instances of IFeature
Set swAnn = swSelMgr.GetSelectedObject6(1, -1)

' Is Flatness1 an annotation or a feature?
If (swAnn.IsDimXpertAnnotation()) Then
    msgStr = "Flatness1 is an annotation"
    Debug.Print ""
    Debug.Print msgStr
ElseIf (swAnn.IsDimXpertFeature()) Then
    msgStr = "Flatness1 is a feature"
    Debug.Print ""
    Debug.Print msgStr
End If

' Get all DimXpert features and annotations using DimXpertPart
Set swDXPart = swSchema.DimXpertPart

' Display all DimXpert annotations using IDimXpertManager.DimXpertPart

Dim annCount As Long
annCount = swDXPart.GetAnnotationCount
msgStr = "Total of "
msgStr2 = annCount
msgStr = msgStr + msgStr2 + " annotations in " + (swSchema.SchemaName)
```

```

Debug.Print ""
Debug.Print msgStr

Dim annotations As Variant
Dim annotation As IDimXpertAnnotation
annotations = swDXPart.GetAnnotations

msgStr = (swSchema.SchemaName) + " has the following annotations: "
Debug.Print ""
Debug.Print msgStr
For n = 0 To UBound(annotations)
    Set annotation = annotations(n)
    Debug.Print " " + (annotation.Name)
Next

' Select another DimXpert feature using swSelectType_e.swSelSWIFTFEATURES type
boolstatus = swModelDocExt.SelectByID2("Plane1", "SWIFTFEATURE", 0, 0, 0, False, 0, Nothing, 0)
Set swFeature = swSelMgr.GetSelectedObject6(1, -1)

' Is Plane1 an annotation or a feature?
If (swFeature.IsDimXpertFeature()) Then
    msgStr = "Plane1 is a feature"
    Debug.Print ""
    Debug.Print msgStr
ElseIf (swFeature.IsDimXpertAnnotation()) Then
    msgStr = "Flatness1 is an annotation"
    Debug.Print msgStr
End If

' Display all DimXpert features using IDimXpertManager.DimXpertPart

Dim featCount As Long
featCount = swDXPart.GetFeatureCount
msgStr = "Total of "
msgStr2 = featCount
msgStr = msgStr + msgStr2 + " features in " + (swSchema.SchemaName)
Debug.Print ""
Debug.Print msgStr

Dim features As Variant
Dim feature As IDimXpertFeature
features = swDXPart.GetFeatures

msgStr = (swSchema.SchemaName) + " has the following features: "
Debug.Print ""
Debug.Print msgStr
For n = 0 To UBound(features)
    Set feature = features(n)
    Debug.Print " " + (feature.Name)
Next

End Sub

```

Get DimXpertManager Info Example (VB.NET)

This example shows the usage of several interfaces that provide access to information on the DimXpertManager tab of the Management Panel in SOLIDWORKS Design.

The IDimXpertManager interface exposes schema information on the DimXpertManager tab.

This macro demonstrates several ways of acquiring IDimXpertManager on an open document:

- ISelectionManager.GetSelectedObject6()
- IModelDocExtension.DimXpertManager
- IConfiguration.DimXpertManager

This macro also demonstrates how to get the features and annotations displayed on the DimXpertManager tab using:

- ISelectionManager.GetSelectedObject6()
- IDimXpertManager.DimXpertPart

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\cover_with_geometric_tolerances.sldprt.
' 2. Select the DimXpertManager tab in the Management Panel.
' 3. Open the Immediate window.
' 4. Add SolidWorks.Interop.swdimxpert.dll as a reference
'    (in the Project Explorer, right-click References, click Add Reference,
'     click Browse, and navigate to install_dir\api\redist).
'
' Postconditions: Inspect the Immediate Window.
'
' NOTE: Because this part is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swdimxpert
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As IModelDoc2
    Dim swModelDocExt As IModelDocExtension
    Dim swSelMgr As ISelectionMgr
    Dim swConfig As IConfiguration
    Dim swFeature As IFeature
    Dim swAnn As IFeature
    Dim swSchema As IDimXpertManager
    Dim swDXPart As DimXpertPart
    Dim msgStr As String
    Dim msgStr2 As String
    Dim displaytype As swDimXpertTreeDisplay_e
    Dim n As Integer
    Dim boolstatus As Boolean

    Sub main()

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager

        ' The following code is commented out because it may not work in early releases of 2010
        ' Select a DimXpert schema using swSelectType_e.swSelSWIFTSHEMA type
        ' boolstatus = swModelDocExt.SelectByID2("Scheme15", "SWIFTSHEMA", 0, 0, 0, False, 0, Nothing, 0)
        ' Get IDimXpertManager (swSchema) for the DimXpert schema using ISelectionManager.GetSelectedObject6()
        ' Set swSchema = swSelMgr.GetSelectedObject6(1, -1)
        ' Debug.Print (swSchema.SchemaName) + " acquired through ISelectionManager"

        ' Get IDimXpertManager (swSchema) for a DimXpert schema using IModelDocExtension.DimXpertManager()
        swSchema = swModelDocExt.DimXpertManager("Default", True)
        Debug.Print((swSchema.SchemaName) + " acquired through IModelDocExtension")

        ' Get IDimXpertManager (swSchema) for a DimXpert schema using IConfiguration.DimXpertManager()
        swConfig = swModel.GetConfigurationByName("Default")
        swSchema = swConfig.DimXpertManager(True)
        Debug.Print((swSchema.SchemaName) + " acquired through IConfiguration")
        msgStr = (swSchema.SchemaName) + " shows a "
        displaytype = swSchema.TreeDisplay
        If (displaytype = swDimXpertTreeDisplay_e.swDimXpertTreeDisplay_Flat) Then
            Debug.Print(msgStr + "flat display of information on DimXpert tab")
        ElseIf (displaytype = swDimXpertTreeDisplay_e.swDimXpertTreeDisplay_Annotation) Then
            Debug.Print(msgStr + "annotation-based display of information on DimXpert tab")
        ElseIf (displaytype = swDimXpertTreeDisplay_e.swDimXpertTreeDisplay_Feature) Then
            Debug.Print(msgStr + "feature-based display of information on DimXpert tab")
        End If

        ' Select a DimXpert annotation using swSelectType_e.swSelSWIFTANOTATIONS type
        boolstatus = swModelDocExt.SelectByID2("Flatness1", "SWIFTANN", 0, 0, 0, False, 0, Nothing, 0)

        ' All DimXpert features and annotations returned by GetSelectedObject6 are instances of IFeature
        swAnn = swSelMgr.GetSelectedObject6(1, -1)

        ' Is Flatness1 an annotation or a feature?
        If (swAnn.IsDimXpertAnnotation()) Then
            msgStr = "Flatness1 is an annotation"
            Debug.Print("")
            Debug.Print(msgStr)
        ElseIf (swAnn.IsDimXpertFeature()) Then
            msgStr = "Flatness1 is a feature"
            Debug.Print("")
            Debug.Print(msgStr)
        End If
    End Sub
End Class

```

```

' Get all DimXpert features and annotations using DimXpertPart
swDXPart = swSchema.DimXpertPart

' Display all DimXpert annotations using IDimXpertManager.DimXpertPart
Dim annCount As Integer
annCount = swDXPart.GetAnnotationCount
msgStr = "Total of "
msgStr2 = annCount
msgStr = msgStr + msgStr2 + " annotations in " + (swSchema.SchemaName)
Debug.Print("")
Debug.Print(msgStr)
Dim annotations As Object
Dim annotation As IDimXpertAnnotation
annotations = swDXPart.GetAnnotations
msgStr = (swSchema.SchemaName) + " has the following annotations: "
Debug.Print("")
Debug.Print(msgStr)
For n = 0 To UBound(annotations)
    annotation = annotations(n)
    Debug.Print(" " + (annotation.Name))
Next

' Select another DimXpert feature using swSelectType.e.swSelSWIFTFEATURES type
boolstatus = swModelDocExt.SelectByID2("Plane1", "SWIFTFEATURE", 0, 0, 0, False, 0, Nothing, 0)
swFeature = swSelMgr.GetSelectedObject6(1, -1)

' Is Plane1 an annotation or a feature?
If (swFeature.IsDimXpertFeature()) Then
    msgStr = "Plane1 is a feature"
    Debug.Print("")
    Debug.Print(msgStr)
ElseIf (swFeature.IsDimXpertAnnotation()) Then
    msgStr = "Flatness1 is an annotation"
    Debug.Print(msgStr)
End If

' Display all DimXpert features using IDimXpertManager.DimXpertPart
Dim featCount As Integer
featCount = swDXPart.GetFeatureCount
msgStr = "Total of "
msgStr2 = featCount
msgStr = msgStr + msgStr2 + " features in " + (swSchema.SchemaName)
Debug.Print("")
Debug.Print(msgStr)
Dim features As Object
Dim feature As IDimXpertFeature
features = swDXPart.GetFeatures
msgStr = (swSchema.SchemaName) + " has the following features: "
Debug.Print("")
Debug.Print(msgStr)
For n = 0 To UBound(features)
    feature = features(n)
    Debug.Print(" " + (feature.Name))
Next
End Sub

Public swApp As SldWorks

End Class

```

Get the Direction of a Bend Example (C#)

This example shows how to get the direction of a bend in sheet metal.

```
//-----
// Preconditions:
// 1. Open a sheet metal part with a bend feature.
// 2. Select the bend feature.
//
// Postconditions: Inspect the Immediate window.
// -----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace GetBendDirection_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        Feature f;

        OneBendFeatureData obe;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            f = (Feature)swModel.ISelectionManager.GetSelectedObject6(1, -1);
            obe = (OneBendFeatureData)f.GetDefinition();
            if (obe.BendDirection == 1)
            {
                Debug.Print("Direction of the selected bend is up.");
            }
            else if (obe.BendDirection == 2)
            {
                Debug.Print("Direction of the selected bend is down.");
            }
            else
            {
                Debug.Print("Error getting the direction of the selected bend.");
            }
        }

        public SldWorks swApp;
    }
}
}
```

Get the Direction of a Bend Example (VBA)

This example shows how to get the direction of a bend in sheet metal.

```
'-----  
' Preconditions:  
' 1. Open a sheet metal part with a bend feature.  
' 2. Select the bend feature.  
'-----  
' Postconditions: Inspect the Immediate window.  
'-----  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim f As SldWorks.Feature  
Dim obe As SldWorks.OneBendFeatureData  
Option Explicit  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set f = swModel.ISelectionManager.GetSelectedObject6(1, -1)  
    Set obe = f.GetDefinition  
    If obe.BendDirection = 1 Then  
        Debug.Print "Direction of the selected bend is up."  
    ElseIf obe.BendDirection = 2 Then  
        Debug.Print "Direction of the selected bend is down."  
    Else  
        Debug.Print "Error getting the direction of the selected bend."  
    End If  
End Sub
```

Get the Direction of a Bend Example (VB.NET)

This example shows how to get the direction of a bend in sheet metal.

```
'-----  
' Preconditions:  
' 1. Open a sheet metal part with a bend feature.  
' 2. Select the bend feature.  
'-----  
' Postconditions: Inspect the Immediate window.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System  
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Dim swModel As ModelDoc2  
    Dim f As Feature  
    Dim obe As OneBendFeatureData  
  
    Sub main()  
  
        swModel = swApp.ActiveDoc  
        f = swModel.ISelectionManager.GetSelectedObject6(1, -1)  
        obe = f.GetDefinition  
        If obe.BendDirection = 1 Then  
            Debug.Print("Direction of the selected bend is up.")  
        ElseIf obe.BendDirection = 2 Then  
            Debug.Print("Direction of the selected bend is down.")  
        Else  
            Debug.Print("Error getting the direction of the selected bend.")  
        End If  
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Get Direction of Bendline Example (C#)

This example shows how to get the direction of the selected bendline.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\2012-sm.sldprt.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Unsuppresses the Flat-Pattern1 feature.
// 2. Selects a bendline.
// 3. Gets the direction of the bendline.
// 4. Examine the Immediate window.
//
// NOTE: Because the part is used elsewhere, do not save
// changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace GetBendLineDirectionSketchLineCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel;
            ModelDocExtension swModelDocExt;
            SelectionMgr swSelMgr;
            SketchLine swSketchLine;
            bool status = false;

            // Open a sheet metal part
            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Select the flat-pattern feature
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);

            // Unsuppress the flat-pattern feature
            status = swModel.EditUnsuppress2();
            swModel.ClearSelection2(true);

            // Select a bendline
            status = swModelDocExt.SelectByID("Line12@Bend-Lines1", "EXTSKETCHSEGMENT", 0.0136749256504085, -0.00842517000103271, 0, false, 0, null, 0);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swSketchLine = (SketchLine)swSelMgr.GetSelectedObject6(1, -1);

            // Print to the Immediate window the direction of the selected bend line
            Debug.Print("Direction of bend line (0 = not a bendline; 1 = bendline has up direction; 2 = bendline has down direction): " + swSketchLine.GetBendLineDirection());
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>

        public SldWorks swApp;
    }
}
```

Get Direction of Bendline Example (VBA)

This example shows how to get the direction of the selected bendline.

```
' -----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\2012-sm.sldprt.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Unsuppresses the Flat-Pattern1 feature.
' 2. Selects a bendline.
' 3. Gets the direction of the bendline.
' 4. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save
' changes.
' -----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swSketchLine As SldWorks.SketchLine
Dim status As Boolean

Sub main()

Set swApp = Application.SldWorks

    ' Open a sheet metal part
    Set swModel = swApp.ActiveDoc

    ' Select the flat-pattern feature
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)

    ' Unsuppress the flat-pattern feature
    status = swModel.EditUnsuppress2
    swModel.ClearSelection2 True

    ' Select a bendline
    status = swModelDocExt.SelectByID("Line12@Bend-Lines1", "EXTSKETCHSEGMENT", 1.36749256504085E-02, -8.42517000103271E-03, 0, False, 0, Nothing, 0)
    Set swSelMgr = swModel.SelectionManager
    Set swSketchLine = swSelMgr.GetSelectedObject6(1, -1)

    ' Print to the Immediate window the direction of the selected bend line
    Debug.Print "Direction of bend line (0= not a bendline; 1 = bendline has up direction; 2 = bendline has down direction): " &
    swSketchLine.GetBendLineDirection

End Sub
```

Get Direction of Bendline Example (VB.NET)

This example shows how to get the direction of the selected bendline.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\2012-sm.sldprt.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Unsuppresses the Flat-Pattern1 feature.
' 2. Selects a bendline.
' 3. Gets the direction of the bendline.
' 4. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save
' changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swSketchLine As SketchLine
        Dim status As Boolean

        ' Open a sheet metal part
        swModel = swApp.ActiveDoc

        ' Select the flat-pattern feature
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)

        ' Unsuppress the flat-pattern feature
        status = swModel.EditUnsuppress2
        swModel.ClearSelection2(True)

        ' Select a bendline
        status = swModelDocExt.SelectByID2("Line12@Bend-Lines1", "EXTSKETCHSEGMENT", 0.0136749256504085, -0.00842517000103271, 0,
False, 0, Nothing, 0)
        swSelMgr = swModel.SelectionManager

        swSketchLine = swSelMgr.GetSelectedObject6(1, -1)

        ' Print to the Immediate window the direction of the selected bend line
        Debug.Print("Direction of bend line (0 = not a bendline; 1 = bendline has up direction; 2 = bendline has down direction): " &
swSketchLine.GetBendLineDirection)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get Dispatch Objects for Selection Set Items Example (C#)

This example shows how to get the Dispatch objects for selection set items.

```
//-----
// Preconditions:
// 1. Verify that the part to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the part.
// 2. Selects two faces and creates a selection set.
// 3. Selects two edges and creates a selection set.
// 4. Traverses the FeatureManager design tree. If the feature is the
//    Selection Sets folder, then:
//    a. Gets its name
//    b. Gets each selection set in the Selection Sets folder.
//       1. Gets the selection set items and their types.
//       2. Gets the Dispatch object for each selection set item.
//       3. Gets the name of the body for each selection set item.
// 5. Examine the Immediate window and FeatureManager design tree.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionSet swSelectionSet1 = default(SelectionSet);
            SelectionSet swSelectionSet2 = default(SelectionSet);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\block.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Create a selection set
            status = swModelDocExt.SelectByID2("", "FACE", -0.0232252739277783, -0.0248041260152831, 0, false, 0, null, 0);
            status = swModelDocExt.SelectByID2("", "FACE", -0.0258449612883283, -0.00628596853454155, -0.00749999999999318, true, 0, null, 0);
            swSelectionSet1 = (SelectionSet)swModelDocExt.SaveSelection(out errors);
            Debug.Print(" First selection set created (1 = succeeded; 0 = failed)? " + errors);
            swModel.ClearSelection2(true);

            //Create another selection set
            status = swModelDocExt.SelectByID2("", "EDGE", 0.0297500073491506, -0.00696186204362448, -0.0073680822854385, false, 0, null, 0);
            status = swModelDocExt.SelectByID2("", "EDGE", 0.0299025466658236, -0.0199243068692567, -0.0036983143110092, true, 0, null, 0);
            swSelectionSet2 = (SelectionSet)swModelDocExt.SaveSelection(out errors);
            Debug.Print(" Second selection set created (1 = succeeded; 0 = failed)? " + errors);
            swModel.ClearSelection2(true);

            //Traverse the model to get Selection Sets folder
            TraverseModelFeatures(swModel);

        }

        public void TraverseFeatureFeatures(Feature swFeat, ModelDoc2 swModel)
        {
            SelectionSetFolder swSelectionSetFolder = default(SelectionSetFolder);
            object[] selectionSetArray = null;
            object[] selectionSetItemArray = null;
            int[] selectionSetItemArrayTypes = null;
            SelectionSet swSelectionSet = default(SelectionSet);
            SelectionSetItem swSelectionSetItem = default(SelectionSetItem);
            Face2 swFace = default(Face2);
            Edge swEdge = default(Edge);
            Body2 swBody = default(Body2);
            int i = 0;
            int j = 0;
            int selectionSetItemArrayType = 0;

            while ((swFeat != null))
            {
                if (swFeat.Name == "Selection Sets")
                {
                    Debug.Print(" " + swFeat.Name + " [" + swFeat.GetTypeName() + "]");
                    //Get Selection Sets folder
                    swSelectionSetFolder = (SelectionSetFolder)swFeat.GetSpecificFeature2();
                    //Get selection sets in Selection Sets folder
                    selectionSetArray = (object[])swSelectionSetFolder.GetSelectionSets();
                    for (i = 0; i < selectionSetArray.GetLength(0); i++)
                    {
                        swSelectionSet = (SelectionSet)selectionSetArray[i];
                        Debug.Print(" Selection set[" + i + "] name: " + swSelectionSet.GetName());
                        //Get the items and their types in this selection set
                        selectionSetItemArray = (object[])swSelectionSet.GetSelectionSetItems();
                        selectionSetItemArrayTypes = (int[])swSelectionSet.GetSelectionSetItemTypes();
                        for (j = 0; j < selectionSetItemArray.GetLength(0); j++)
                        {
                            swSelectionSetItem = (SelectionSetItem)selectionSetItemArray[j];
                            selectionSetItemArrayType = (int)selectionSetItemArrayTypes[j];
                            switch (selectionSetItemArrayType)
                            {
                                case (int)swSelectType_e.swSelFACES:
                                    //Get the Dispatch object for the selection set item
                                    swFace = (Face2)swSelectionSetItem.GetCorrespondingItem();
                                    //Get the name of the body for the face

```

```

        swBody = (Body2)swFace.GetBody();
        Debug.Print("        Name of face[" + j + "]"s body: " + swBody.Name);
        break;
    case (int)swSelectType_e.swSeLEDGES:
        //Get the Dispatch object for the selection set item
        swEdge = (Edge)swSelectionSetItem.GetCorrespondingItem();
        //Get the name of the body for the edge
        swBody = (Body2)swEdge.GetBody();
        Debug.Print("        Name of edge[" + j + "]"s body: " + swBody.Name);
        break;
    }
}
}
}
}
    swFeat = (Feature)swFeat.GetNextFeature();
}
}

public void TraverseModelFeatures(ModelDoc2 swModel)
{
    Feature swFeat = default(Feature);
    swFeat = (Feature)swModel.FirstFeature();
    TraverseFeatureFeatures(swFeat, swModel);
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Get Dispatch Objects for Selection Set Items Example (VBA)

This example shows how to get the Dispatch objects for selection set items.

```
'-----
' Preconditions:
' 1. Verify that the part to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the part.
' 2. Selects two faces and creates a selection set.
' 3. Selects two edges and creates a selection set.
' 4. Traverses the FeatureManager design tree. If the feature is the
'    Selection Sets folder, then:
'    a. Gets its name.
'    b. Gets each selection set in the Selection Sets folder.
'       1. Gets the selection set items and their types.
'       2. Gets the Dispatch object for each selection set item.
'       3. Gets the name of the body for each selection set item.
' 5. Examine the Immediate window and FeatureManager design tree.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelectionSet1 As SldWorks.SelectionSet
Dim swSelectionSet2 As SldWorks.SelectionSet
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String

Sub TraverseFeatureFeatures(swFeat As SldWorks.Feature, swModel As SldWorks.ModelDoc2)
    Dim swSelectionSetFolder As SldWorks.SelectionSetFolder
    Dim selectionSetArray As Variant
    Dim selectionSetItemArray As Variant
    Dim selectionSetItemArrayTypes As Variant
    Dim swSelectionSet As SldWorks.SelectionSet
    Dim swSelectionSetItem As SldWorks.SelectionSetItem
    Dim swFace As SldWorks.Face2
    Dim swEdge As SldWorks.Edge
    Dim swBody As SldWorks.Body2
    Dim i As Long
    Dim j As Long

    While Not swFeat Is Nothing
        If swFeat.name = "Selection Sets" Then
            Debug.Print "    " & swFeat.name & " [" & swFeat.GetTypeName & "]"
            'Get Selection Sets folder
            Set swSelectionSetFolder = swFeat.GetSpecificFeature2
            'Get selection sets in Selection Sets folder
            selectionSetArray = swSelectionSetFolder.GetSelectionSets
            For i = 0 To UBound(selectionSetArray)
                Set swSelectionSet = selectionSetArray(i)
                Debug.Print "        Selection set[" & i & "] name: " & swSelectionSet.GetName
                'Get the items and their types in this selection set
                selectionSetItemArray = swSelectionSet.GetSelectionSetItems
                selectionSetItemArrayTypes = swSelectionSet.GetSelectionSetItemTypes
                For j = 0 To UBound(selectionSetItemArray)
                    Set swSelectionSetItem = selectionSetItemArray(j)
                    Select Case selectionSetItemArrayTypes(j)
                        Case swSelectType_e.swSelFACES
                            'Get the Dispatch object for the selection set item
                            Set swFace = swSelectionSetItem.GetCorrespondingItem
                            'Get the name of the body for the face
                            Set swBody = swFace.GetBody
                            Debug.Print "            Name of face[" & j & "]'s body: " & swBody.name
                        Case swSelectType_e.swSelEDGES
                            'Get the Dispatch object for the selection set item
                            Set swEdge = swSelectionSetItem.GetCorrespondingItem
                            'Get the name of the body for the edge
                            Set swBody = swEdge.GetBody
                            Debug.Print "            Name of edge[" & j & "]'s body: " & swBody.name
                    End Select
                Next
            Next
            Set swFeat = swFeat.GetNextFeature
        End If
    Wend
End Sub

Sub TraverseModelFeatures(swModel As SldWorks.ModelDoc2)
    Dim swFeat As SldWorks.Feature

    Set swFeat = swModel.FirstFeature
    TraverseFeatureFeatures swFeat, swModel
End Sub

Sub main()
```

```

Set swApp = Application.SldWorks
fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block.sldprt"
Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
Set swModelDocExt = swModel.Extension

'Create a selection set
status = swModelDocExt.SelectByID("", "FACE", -2.32252739277783E-02, -2.48041260152831E-02, 0, False, 0, Nothing, 0)
status = swModelDocExt.SelectByID("", "FACE", -2.58449612883283E-02, -6.28596853454155E-03, -7.49999999999318E-03, True, 0, Nothing, 0)
Set swSelectionSet1 = swModelDocExt.SaveSelection(errors)
Debug.Print " First selection set created (1 = succeeded; 0 = failed)? " & errors
swModel.ClearSelection2 True

'Create another selection set
status = swModelDocExt.SelectByID("", "EDGE", 2.97500073491506E-02, -6.96186204362448E-03, -7.3680822854385E-03, False, 0, Nothing, 0)
status = swModelDocExt.SelectByID("", "EDGE", 2.99025466658236E-02, -1.99243068692567E-02, -3.6983143110092E-03, True, 0, Nothing, 0)
Set swSelectionSet2 = swModelDocExt.SaveSelection(errors)
Debug.Print " Second selection set created (1 = succeeded; 0 = failed)? " & errors
swModel.ClearSelection2 True

'Traverse the model to get Selection Sets folder
TraverseModelFeatures swModel

End Sub

```

Get Dispatch Objects for Selection Set Items Example (VB.NET)

This example shows how to get the Dispatch objects for selection set items.

```

-----
Preconditions:
1. Verify that the part to open exists.
2. Open the Immediate window.

Postconditions:
1. Opens the part.
2. Selects two faces and creates a selection set.
3. Selects two edges and creates a selection set.
4. Traverses the FeatureManager design tree. If the feature is the
   Selection Sets folder, then:
   a. Gets its name.
   b. Gets each selection set in the Selection Sets folder.
      1. Gets the selection set items and their types.
      2. Gets the Dispatch object for each selection set item.
      3. Gets the name of the body for each selection set item.
5. Examine the Immediate window and FeatureManager design tree.

NOTE: Because the part is used elsewhere, do not save changes.
-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelectionSet1 As SelectionSet
        Dim swSelectionSet2 As SelectionSet
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension

        'Create a selection set
        status = swModelDocExt.SelectByID2("", "FACE", -0.0232252739277783, -0.0248041260152831, 0, False, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "FACE", -0.0258449612883283, -0.00628596853454155, -0.007499999999999318, True, 0, Nothing, 0)
        swSelectionSet1 = swModelDocExt.SaveSelection(errors)
        Debug.Print(" First selection set created (1 = succeeded; 0 = failed)? " & errors)
        swModel.ClearSelection2(True)

        'Create another selection set
        status = swModelDocExt.SelectByID2("", "EDGE", 0.0297500073491506, -0.00696186204362448, -0.0073680822854385, False, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "EDGE", 0.0299025466658236, -0.0199243068692567, -0.0036983143110092, True, 0, Nothing, 0)
        swSelectionSet2 = swModelDocExt.SaveSelection(errors)
        Debug.Print(" Second selection set created (1 = succeeded; 0 = failed)? " & errors)
        swModel.ClearSelection2(True)

        'Traverse the model to get the Selection Sets folder
        TraverseModelFeatures(swModel)

    End Sub

    Sub TraverseFeatureFeatures(ByVal swFeat As Feature, ByVal swModel As ModelDoc2)
        Dim swSelectionSetFolder As SelectionSetFolder
        Dim selectionSetArray As Object
        Dim selectionSetItemArray As Object
        Dim selectionSetItemArrayTypes As Object
        Dim swSelectionSet As SelectionSet
        Dim swSelectionSetItem As SelectionSetItem
        Dim swFace As Face2
        Dim swEdge As Edge
        Dim swBody As Body2
        Dim i As Integer
        Dim j As Integer
        Dim selectionSetItemArrayType As Integer

        While Not swFeat Is Nothing
            If swFeat.name = "Selection Sets" Then
                Debug.Print(" " & swFeat.name & " [" & swFeat.GetTypeName & "]" )
                'Get Selection Sets folder
                swSelectionSetFolder = swFeat.GetSpecificFeature2
                'Get selection sets in Selection Sets folder
                selectionSetArray = swSelectionSetFolder.GetSelectionSets
                For i = 0 To UBound(selectionSetArray)
                    swSelectionSet = selectionSetArray(i)
                    Debug.Print(" Selection set[" & i & "]" name: " & swSelectionSet.GetName)
                    'Get the items and their types in this selection set
                    selectionSetItemArray = swSelectionSet.GetSelectionSetItems
                    selectionSetItemArrayTypes = swSelectionSet.GetSelectionSetItemTypes
                    For j = 0 To UBound(selectionSetItemArray)
                        swSelectionSetItem = selectionSetItemArray(j)
                        selectionSetItemArrayType = selectionSetItemArrayTypes(j)
                        Select Case selectionSetItemArrayType
                            Case swSelectType_e.swSelFACES
                                'Get the Dispatch object for the selection set item
                                swFace = swSelectionSetItem.GetCorrespondingItem
                                'Get the name of the body for the face
                                swBody = swFace.GetBody
                                Debug.Print(" Name of face[" & j & "]"'s body: " & swBody.Name)
                            Case swSelectType_e.swSelEDGES
                                'Get the Dispatch object for the selection set item
                                swEdge = swSelectionSetItem.GetCorrespondingItem
                                'Get the name of the body for the edge
                                swBody = swEdge.GetBody
                                Debug.Print(" Name of edge[" & j & "]"'s body: " & swBody.Name)
                        End Select
                    Next
                Next
            End If
        End While
    End Sub

```

```

        Next
    End If
    swFeat = swFeat.NextFeature
End While
End Sub

Sub TraverseModelFeatures(ByVal swModel As ModelDoc2)
    Dim swFeat As Feature
    swFeat = swModel.FirstFeature
    TraverseFeatureFeatures(swFeat, swModel)
End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Get Display Dimension Properties Examples (VBA)

This example shows how to get the display dimension properties for a dimension in a drawing view.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Iterates through the display dimensions in DrawingView1.
' 2. If the display dimension's name is RD19, then
'    gets its properties.
' 3. Examine the Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not make changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDrawingDoc As SldWorks.DrawingDoc
Dim swView As SldWorks.View
Dim swDrawingView1 As SldWorks.View
Dim swDisplayDimension As SldWorks.DisplayDimension
Dim swDimension As SldWorks.Dimension

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swDrawingDoc = swModel
    'First view is sheet
    Set swView = swDrawingDoc.GetFirstView
    'So, get next view, which is DrawingView1
    Set swDrawingView1 = swView.GetNextView

    'Iterate through the display dimensions in DrawingView1
    Set swDisplayDimension = swDrawingView1.GetFirstDisplayDimension()
    While Not swDisplayDimension Is Nothing
        'Check if display dimension is RD19
        Set swDimension = swDisplayDimension.GetDimension
        If swDimension.Name = "RD19" Then
            'Get the properties of display dimension RD19
            Debug.Print "Display dimension RD19's properties:"
            Debug.Print " Arrow side: " & swDisplayDimension.ArrowSide
            Debug.Print " Broken leader: " & swDisplayDimension.BrokenLeader
            Debug.Print " Leader visibility: "; swDisplayDimension.LeaderVisibility
            Debug.Print " Show dimension value: " & swDisplayDimension.ShowDimensionValue
            Debug.Print " Show parenthesis: " & swDisplayDimension.ShowParenthesis
            Debug.Print " Smart witness: " & swDisplayDimension.SmartWitness
            Debug.Print " Witness visibility: " & swDisplayDimension.WitnessVisibility
        End If
        Set swDisplayDimension = swDisplayDimension.GetNext()
    ' Get next display dimension
    Wend

End Sub
```

Get Display Dimensions, GTols, and Surface-Finish Symbols Example (VBA)

This example shows how to get all of the annotations in a part, assembly, or drawing.

```
'-----
' Preconditions:
' 1. Open a part, assembly, or drawing document.
' 2. Open the Immediate window.
' 3. Run the macro.
'
' Postconditions:
' 1. Prints any annotations to the Immediate window.
' 2. Examine the Immediate window.
'-----
```

Option Explicit

```
Sub ProcessAnnotation(swApp As SldWorks.SldWorks, swAnn As SldWorks.Annotation)
    Dim swAnnCThread As SldWorks.CThread
    Dim swAnnDatumTag As SldWorks.DatumTag
    Dim swAnnDatumTargetSym As SldWorks.DatumTargetSym
    Dim swAnnDisplayDimension As SldWorks.DisplayDimension
    Dim swAnnGTol As SldWorks.Gtol
    Dim swAnnNote As SldWorks.Note
    Dim swAnnSFSymbol As SldWorks.SFSymbol
    Dim swAnnWeldSymbol As SldWorks.WeldSymbol
    Dim swAnnCustomSymbol As SldWorks.CustomSymbol
    Dim swAnnDowelSym As SldWorks.DowelSymbol
    Dim swAnnLeader As SldWorks.MultiJogLeader
    Dim swAnnCenterMarkSym As SldWorks.CenterMark
    Dim swAnnTable As SldWorks.TableAnnotation
    Dim swAnnCenterLine As SldWorks.Centerline
    Dim swAnnDatumOrigin As SldWorks.DatumOrigin

    Select Case swAnn.GetType
        Case swCThread
            Set swAnnCThread = swAnn.GetSpecificAnnotation
            Debug.Print " Annotation type (enumerator): swAnnotationType_e.swCThread"
            Debug.Print ""
        Case swDatumTag
            Set swAnnDatumTag = swAnn.GetSpecificAnnotation
            Debug.Print " Annotation type (enumerator): swAnnotationType_e.swDatumTag"
            Debug.Print ""
        Case swDatumTargetSym
            Set swAnnDatumTargetSym = swAnn.GetSpecificAnnotation
            Debug.Print " Annotation type (enumerator): swAnnotationType_e.swDatumTargetSym"
            Debug.Print ""
        Case swDisplayDimension
            Set swAnnDisplayDimension = swAnn.GetSpecificAnnotation
            Debug.Print " Annotation type (enumerator): swAnnotationType_e.swDisplayDimension"
            Debug.Print ""
        Case swGTol
            Set swAnnGTol = swAnn.GetSpecificAnnotation
            Debug.Print " Annotation type (enumerator): swAnnotationType_e.swGTol"
            Debug.Print ""
        Case swNote
            Set swAnnNote = swAnn.GetSpecificAnnotation
            Debug.Print " Annotation type (enumerator): swAnnotationType_e.swNote"
            Debug.Print ""
        Case swSFSymbol
            Set swAnnSFSymbol = swAnn.GetSpecificAnnotation
            Debug.Print " Annotation type (enumerator): swAnnotationType_e.swSFSymbol"
            Debug.Print ""
        Case swWeldSymbol
            Set swAnnWeldSymbol = swAnn.GetSpecificAnnotation
            Debug.Print " Annotation type (enumerator): swAnnotationType_e.swWeldSymbol"
            Debug.Print ""
        Case swCustomSymbol
            Set swAnnCustomSymbol = swAnn.GetSpecificAnnotation
            Debug.Print " Annotation type (enumerator): swAnnotationType_e.swCustomSymbol"
            Debug.Print ""
        Case swDowelSym
            Set swAnnDowelSym = swAnn.GetSpecificAnnotation
            Debug.Print " Annotation type (enumerator): swAnnotationType_e.swDowelSym"
            Debug.Print ""
        Case swLeader
            Set swAnnLeader = swAnn.GetSpecificAnnotation
            Debug.Print " Annotation type (enumerator): swAnnotationType_e.swLeader"
            Debug.Print ""
        Case swCenterMarkSym
            Set swAnnCenterMarkSym = swAnn.GetSpecificAnnotation
            Debug.Print " Annotation type (enumerator): swAnnotationType_e.swCenterMarkSym"
            Debug.Print ""
        Case swTableAnnotation
            Set swAnnTable = swAnn.GetSpecificAnnotation
            Debug.Print " Annotation type (enumerator): swAnnotationType_e.swTableAnnotation"
            Debug.Print ""
        Case swCenterLine
            Set swAnnCenterLine = swAnn.GetSpecificAnnotation
            Debug.Print " Annotation type (enumerator): swAnnotationType_e.swCenterLine"
            Debug.Print ""
        Case swDatumOrigin
            Set swAnnDatumOrigin = swAnn.GetSpecificAnnotation
            Debug.Print " Annotation type (enumerator): swAnnotationType_e.swDatumOrigin"
            Debug.Print ""
    End Select
End Sub
```

```

        Case Else
            Debug.Print " Unknown annotation type"
            Debug.Print ""
            Debug.Assert False
        End Select
    End Sub

Sub ProcessModel(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2)
    Dim swAnn As SldWorks.Annotation
    Dim nNumLeader As Long
    Dim nNumPts As Long
    Dim vLeaderPt As Variant
    Dim i As Long
    Dim j As Long
    Dim k As Long
    Dim bRet As Boolean

    Debug.Print "Model path and file name: " & swModel.GetPathName
    Debug.Print ""
    Set swAnn = swModel.GetFirstAnnotation2

    Do While Not swAnn Is Nothing
        Debug.Print " Annotation name: " & swAnn.GetName
        Debug.Print " Annotation type (enumerator numeric value): " & swAnn.GetType

        If True = swAnn.GetLeader Then
            Dim nbrLeaders As Long
            nbrLeaders = swAnn.GetLeaderCount
            For i = 0 To swAnn.GetLeaderCount - 1
                If True = swAnn.GetBentLeader Then
                    nNumPts = 3
                Else
                    nNumPts = 2
                End If

                vLeaderPt = swAnn.GetLeaderPointsAtIndex(i)
                If Not IsEmpty(vLeaderPt) Then
                    Dim leaderPt() As Double
                    ReDim leaderPt(nNumPts) As Double
                    leaderPt = vLeaderPt
                    k = 0
                    For j = 0 To UBound(leaderPt)
                        Debug.Print " Pt[" & k & "] x,y,z coordinates:"
                        Debug.Print " " & Str(leaderPt(j)) & ", " & Str(leaderPt(j + 1)) & Str(leaderPt(j + 2))
                        j = j + 2
                        k = k + 1
                    Next j
                End If
            Next i

            End If

            ProcessAnnotation swApp, swAnn

            Set swAnn = swAnn.GetNext3
        Loop
    End Sub

Sub ProcessComponent(swApp As SldWorks.SldWorks, swComp As SldWorks.Component2)
    Dim vChildArray As Variant
    Dim swChildComp As SldWorks.Component2
    Dim swModel As SldWorks.ModelDoc2
    Dim swPart As SldWorks.PartDoc
    Dim i As Long

    vChildArray = swComp.GetChildren

    For i = 0 To UBound(vChildArray)
        Set swChildComp = vChildArray(i)
        ProcessComponent swApp, swChildComp
    Next i

    Set swModel = swComp.GetModelDoc

    If Not swModel Is Nothing Then
        If swDocPART = swModel.GetType Then
            ProcessModel swApp, swModel
        End If
    End If

End Sub

Sub ProcessDrawing(swApp As SldWorks.SldWorks, swDraw As SldWorks.DrawingDoc)
    Dim swView As SldWorks.View
    Dim swAnn As SldWorks.Annotation

    Set swView = swDraw.GetFirstView
    Do While Not Nothing Is swView
        Set swAnn = swView.GetFirstAnnotation3
        Do While Not Nothing Is swAnn
            ProcessAnnotation swApp, swAnn
            Set swAnn = swAnn.GetNext3
        Loop

        Set swView = swView.GetNextView
    Loop

End Sub

Sub main()

```

```

Dim swApp                As SldWorks.SldWorks
Dim swModel              As SldWorks.ModelDoc2
Dim swAssy              As SldWorks.AssemblyDoc
Dim swDraw              As SldWorks.DrawingDoc
Dim swConfig            As SldWorks.Configuration
Dim swConfigMgr         As SldWorks.ConfigurationManager
Dim swRootComp          As SldWorks.Component2
Dim nStatus             As Long
Dim bRet                As Boolean

Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc

Select Case swModel.GetType
    Case swDocPART
        ProcessModel swApp, swModel
    Case swDocASSEMBLY
        Set swAssy = swModel
        nStatus = swAssy.ResolveAllLightWeightComponents(False)
        Set swConfigMgr = swModel.ConfigurationManager
        Set swConfig = swConfigMgr.ActiveConfiguration
        Set swRootComp = swConfig.GetRootComponent
        ProcessComponent swApp, swRootComp
    Case swDocDRAWING
        Set swDraw = swModel
        ProcessDrawing swApp, swDraw
    Case Else
        Exit Sub
End Select

End Sub

```

Get Display State Example (VBA)

This example shows how to get the display state of the model.

```
'-----  
'  
' Preconditions: Model document is open.  
'  
' Postconditions: None  
'  
'-----  
  
Option Explicit  
Public Enum swViewDisplayType_e  
    swlViewSectioned = 0  
    swlViewPerspective = 1  
    swlViewShaded = 2  
    swlViewWireFrame = 3  
    swlViewHiddenLinesRemoved = 4  
    swlViewHiddenInGrey = 5  
    swlViewCurvature = 6  
End Enum  
Sub main()  
    Dim swApp           As SldWorks.SldWorks  
    Dim swModel         As SldWorks.ModelDoc2  
    Dim swModView       As SldWorks.ModelView  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModView = swModel.ActiveView  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print " Sectioned      = " & swModView.GetDisplayState(swlViewSectioned)  
    Debug.Print " Perspective    = " & swModView.GetDisplayState(swlViewPerspective)  
    Debug.Print " Shaded        = " & swModView.GetDisplayState(swlViewShaded)  
    Debug.Print " WireFrame     = " & swModView.GetDisplayState(swlViewWireFrame)  
    Debug.Print " HiddenLinesRemoved = " & swModView.GetDisplayState(swlViewHiddenLinesRemoved)  
    Debug.Print " HiddenInGrey   = " & swModView.GetDisplayState(swlViewHiddenInGrey)  
    Debug.Print " Curvature     = " & swModView.GetDisplayState(swlViewCurvature)  
End Sub  
'-----
```

Get Display State for Each Drawing View Example (C#)

This example shows how to get the display state for each drawing view.

```
//-----
// Preconditions:
// 1. Open a drawing.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Traverses the drawing views on the current sheet and
//    gets each drawing view's display state.
// 2. Examine the Immediate window.
//
// NOTE: Because the drawing is used elsewhere, do not
// save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            DrawingDoc swDraw = default(DrawingDoc);
            Sheet swSheet = default(Sheet);
            View swView = default(View);

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swDraw = (DrawingDoc)swModel;
            swSheet = (Sheet)swDraw.GetCurrentSheet();
            swView = (View)swDraw.GetFirstView();
            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print(" " + swSheet.GetName());
            while ((swView != null))
            {
                Debug.Print(" " + swView.GetName2() + " [" + swView.DisplayState + "]");
                swView = (View)swView.GetNextView();
            }
        }
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
```

Get Display State for Each Drawing View Example (VBA)

This example shows how to get the display state for each drawing view.

```
'-----  
' Preconditions:  
' 1. Open a drawing.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Traverses the drawing views on the current sheet and  
'    gets each drawing view's display state.  
' 2. Examine the Immediate window.  
'  
' NOTE: Because the drawing is used elsewhere, do not  
' save changes.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swDraw As SldWorks.DrawingDoc  
    Dim swSheet As SldWorks.Sheet  
    Dim swView As SldWorks.View  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set swDraw = swModel  
    Set swSheet = swDraw.GetCurrentSheet  
    Set swView = swDraw.GetFirstView  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print " " & swSheet.GetName  
  
    While Not swView Is Nothing  
        Debug.Print " " & swView.GetName2 & " [" & swView.DisplayState & "]"  
        Set swView = swView.GetNextView  
    Wend  
  
End Sub
```

Get Display State for Each Drawing View Example (VB.NET)

This example shows how to get the display state for each drawing view.

```
'-----
' Preconditions:
' 1. Open a drawing.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Traverses the drawing views on the current sheet and
'    gets each drawing view's display state.
' 2. Examine the Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not
'       save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Sub main()

        Dim swModel As ModelDoc2
        Dim swDraw As DrawingDoc
        Dim swSheet As Sheet
        Dim swView As View

        swModel = swApp.ActiveDoc
        swDraw = swModel
        swSheet = swDraw.GetCurrentSheet
        swView = swDraw.GetFirstView
        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print(" " & swSheet.GetName)
        While Not swView Is Nothing
            Debug.Print(" " & swView.GetName2 & " [" & swView.DisplayState & "]")
            swView = swView.GetNextView
        End While
    End Sub

    Public swApp As SldWorks

End Class
```

Get Display State Names and Visibilities of Components Example (C#)

This example shows how to get the names of the display states in an assembly and the visibilities of the assembly components in each display state.

```
//-----
// Preconditions:
// 1. Open an assembly that contains multiple components
// and display states.
// 2. Open the Immediate window.
//
// Postconditions: Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace DisplayStateComponentsCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            ConfigurationManager swConfigMgr = default(ConfigurationManager);
            Configuration swConfig = default(Configuration);
            object oComponents = null;
            object[] components = null;
            Component2 comp = null;
            int docType = 0;
            int i = 0;
            int j = 0;
            string[] displayStateNames = null;
            string displayStateName = null;
            int[] displayStateVisibilities = null;
            int displayStateVisibility = 0;
            string visibility = "";

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            docType = swModel.GetType();
            if (docType == (int)swDocumentTypes_e.swDocASSEMBLY)
            {
                swConfigMgr = (ConfigurationManager)swModel.ConfigurationManager;
                swConfig = (Configuration)swConfigMgr.ActiveConfiguration;
                displayStateNames = (string[])swConfig.GetDisplayStates();
                for (i = 0; i < displayStateNames.Length; i++)
                {
                    displayStateName = (string)displayStateNames[i];
                    Debug.Print("Display state name: " + displayStateName);
                    displayStateVisibilities = (int[])swConfig.GetDisplayStateComponentVisibility(displayStateName, false, false, oComponents);
                    components = (object[])oComponents;
                    Debug.Print("Display state visibility: ");
                    for (j = 0; j < displayStateVisibilities.Length; j++)
                    {
                        displayStateVisibility = (int)displayStateVisibilities[j];
                        switch (displayStateVisibility)
                        {
                            case 0:
                                visibility = "Hidden";
                                break;
                            case 1:
                                visibility = "Shown";
                                break;
                        }
                        comp = (Component2)components[j];
                        Debug.Print(" " + comp.Name2 + ": " + visibility);
                    }
                }
            }
            else
            {
                Debug.Print("Open an assembly document with multiple display states.");
                return;
            }
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
```


Get Display State Names and Visibilities of Components Example (VBA)

This example shows how to get the names of the display states in an assembly and the visibilities of the assembly components in each display state.

```

'-----
' Preconditions:
' 1. Open an assembly that contains multiple components
'    and display states.
' 2. Open the Immediate window.
'
' Postconditions: Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swConfigMgr As SldWorks.ConfigurationManager
Dim swConfig As SldWorks.Configuration
Dim components As Variant
Dim docType As Long
Dim i As Long
Dim j As Long
Dim displayStateNames As Variant
Dim displayStateName As String
Dim displayStateVisibilities As Variant
Dim visibility As String

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    docType = swModel.GetType
    If docType = swDocumentTypes_e.swDocASSEMBLY Then

        Set swConfigMgr = swModel.ConfigurationManager
        Set swConfig = swConfigMgr.ActiveConfiguration
        displayStateNames = swConfig.GetDisplayStates
        For i = 0 To UBound(displayStateNames)
            displayStateName = displayStateNames(i)
            Debug.Print "Display state name: " & displayStateName
            displayStateVisibilities = swConfig.GetDisplayStateComponentVisibility(displayStateName, False, False, components)
            Debug.Print (" Display state visibility: ")
            For j = 0 To UBound(displayStateVisibilities)
                Select Case displayStateVisibilities(j)
                    Case 0
                        visibility = "Hidden"
                    Case 1
                        visibility = "Shown"
                End Select
                Debug.Print ("    " & components(j).Name2 & ": " & visibility)
            Next j
        Next i
    Else
        Debug.Print "Open an assembly document with multiple display states."
        Exit Sub
    End If
End Sub

```

Get Display State Names and Visibilities of Components Example (VB.NET)

This example shows how to get the names of the display states in an assembly and the visibilities of the assembly components in each display state.

```

'-----
' Preconditions:
' 1. Open an assembly that contains multiple components
'    and display states.
' 2. Open the Immediate window.
' Postconditions: Examine the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swConfigMgr As ConfigurationManager
        Dim swConfig As Configuration
        Dim components As Object = Nothing
        Dim docType As Integer
        Dim i As Integer
        Dim j As Integer
        Dim displayStateNames As Object
        Dim displayStateName As String
        Dim displayStateVisibilities As Object
        Dim displayStateVisibility As Integer
        Dim visibility As String = ""

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        docType = swModel.GetType
        If docType = swDocumentTypes_e.swDocASSEMBLY Then
            swConfigMgr = swModel.ConfigurationManager
            swConfig = swConfigMgr.ActiveConfiguration
            displayStateNames = swConfig.GetDisplayStates
            For i = 0 To UBound(displayStateNames)
                displayStateName = displayStateNames(i)
                Debug.Print("Display state name: " & displayStateName)
                displayStateVisibilities = swConfig.GetDisplayStateComponentVisibility(displayStateName, False, False, components)
                Debug.Print("  Display state visibility: ")
                For j = 0 To UBound(displayStateVisibilities)
                    displayStateVisibility = displayStateVisibilities(j)
                    Select Case displayStateVisibility
                        Case 0
                            visibility = "Hidden"
                        Case 1
                            visibility = "Shown"
                    End Select
                    Debug.Print("    " & components(j).Name2 & ": " & visibility)
                Next j
            Next i
        Else
            Debug.Print("Open an assembly document with multiple display states.")
            Exit Sub
        End If

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get Display State Settings Example (C#)

This example shows how to get display modes, transparency states, and visibility states of components.

```
//-----
// Preconditions: Open an assembly that contains a minimum of three top-level
// components and two display states, "DS_1" and "DS_2".
//
// Postconditions: Inspect the Immediate Window for the display modes,
// transparency states, and visibility states of all three components
// in both DS_1 and DS_2.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace GetDisplayMode.csproj
{
    public partial class SolidWorksMacro
    {
        ModelDoc2 swDoc = null;
        ModelDocExtension swExt = null;
        DisplayStateSetting swDSS = null;
        object varStatus;
        object varTStatus;
        object varVStatus;
        Array statusArray;
        Array statusTArray;
        Array statusVArray;
        const int maxEntMode = 3;

        public void Main()
        {
            swDoc = (ModelDoc2)(swApp.ActiveDoc);
            swExt = swDoc.Extension;
            int docType = swDoc.GetType();
            if (docType == (int)swDocumentTypes_e.swDocASSEMBLY)
            {
                CreateDisplayStateSetting();

                varStatus = swExt.get_DisplayMode(swDSS);
                statusArray = (Array)varStatus;

                varTStatus = swExt.get_Transparency(swDSS);
                statusTArray = (Array)varTStatus;

                varVStatus = swExt.get_Visibility(swDSS);
                statusVArray = (Array)varVStatus;

                WriteOutput();
            }
        }

        public void CreateDisplayStateSetting()
        {
            swDSS = swExt.GetDisplayStateSetting((int)swDisplayStateOpts_e.swThisDisplayState);
            swDSS.Option = (int)swDisplayStateOpts_e.swSpecifyDisplayState;

            string[] specDSNames = new string[2];
            specDSNames[0] = "DS_1";
            specDSNames[1] = "DS_2";
            object varSpecDSNames = specDSNames;
            swDSS.Names = varSpecDSNames;

            AssemblyDoc swADoc;
            swADoc = (AssemblyDoc)swDoc;
            int compCnt = swADoc.GetComponentCount(true);
            Component2[] listComp = new Component2[maxEntMode];
            if (compCnt >= maxEntMode)
            {
                object[] varComp = (object[])(swADoc.GetComponents(true));
                listComp[0] = (Component2)varComp[0];
                listComp[1] = (Component2)varComp[1];
                listComp[2] = (Component2)varComp[2];
                swDSS.Entities = listComp;
            }
        }

        public void WriteOutput()
        {
            int entCount = swDSS.GetEntityCount();
            object[] listComp = (object[])swDSS.Entities;
            int allCtr = 0;
            for (int entctr = 0; entctr < entCount; ++entctr)
            {
                Component2 swComp = (Component2)listComp[entctr];
                Debug.Print(swComp.Name2);
                int dsNameCount = swDSS.GetNameCount();
            }
        }
    }
}
```

```

        object[] dsNames = (object[])swDSS.Names;

        for (int namectr = 0; namectr < dsNameCount; ++namectr)
        {
            Debug.Print(" " + (string)dsNames[namectr]);
            int status = (int)statusArray.GetValue(allCtr);
            int statusT = (int)statusTArray.GetValue(allCtr);
            int statusV = (int)statusVArray.GetValue(allCtr);
            WriteMode(status);
            WriteTransparency(statusT);
            WriteVisibility(statusV);
            ++allCtr;
        }
    }
}

public void WriteMode(int status)
{
    switch (status)
    {
        case (int)swDisplayMode_e.swDisplayModeDEFAULT:
            Debug.Print(" swDisplayModeDEFAULT");
            break;
        case (int)swDisplayMode_e.swHIDDEN:
            Debug.Print(" swHIDDEN");
            break;
        case (int)swDisplayMode_e.swHIDDEN_GREYED:
            Debug.Print(" swHIDDEN_GREYED");
            break;
        case (int)swDisplayMode_e.swSHADED:
            Debug.Print(" swSHADED");
            break;
        case (int)swDisplayMode_e.swSHADED_EDGES:
            Debug.Print(" swSHADED_EDGES");
            break;
        case (int)swDisplayMode_e.swWIREFRAME:
            Debug.Print(" swWIREFRAME");
            break;
        case (int)swDisplayMode_e.swDisplayModeUNKNOWN:
            Debug.Print(" Error:swDisplayModeUNKNOWN");
            break;
        case CInt(swDisplayMode_e.swFACETED_WIREFRAME):
            Debug.Print(" swFACETED_WIREFRAME");
            break;
        case CInt(swDisplayMode_e.swFACETED_HIDDEN_GREYED):
            Debug.Print(" swFACETED_HIDDEN_GREYED");
            break;
        case CInt(swDisplayMode_e.swFACETED_HIDDEN):
            Debug.Print(" swFACETED_HIDDEN");
            break;
    }
}

public void WriteTransparency(int status)
{
    switch (status)
    {
        case (int)swTransparencyState_e.swTransparencyStateTransparent:
            Debug.Print(" swTransparencyStateTransparent");
            break;
        case (int)swTransparencyState_e.swTransparencyStateNonTransparent:
            Debug.Print(" swTransparencyStateNonTransparent");
            break;
        case (int)swTransparencyState_e.swTransparencyStateUnknown:
            Debug.Print(" ERROR : swTransparencyStateUnknown");
            break;
    }
}

public void WriteVisibility(int status)
{
    switch (status)
    {
        case (int)swVisibilityState_e.swVisibilityStateHide:
            Debug.Print(" swVisibilityStateHide");
            break;
        case (int)swVisibilityState_e.swVisibilityStateShown:
            Debug.Print(" swVisibilityStateShown");
            break;
        case (int)swVisibilityState_e.swVisibilityStateUnknown:
            Debug.Print(" ERROR : swVisibilityStateUnknown");
            break;
    }
}
}

public SldWorks swApp;
}
}

```

Get Display State Settings Example (VBA)

This example shows how to get display modes, transparency states, and visibility states of components.

```

'-----
' Preconditions: Open an assembly that contains a minimum of three top-level
' components and two display states, "DS_1" and "DS_2".
'-----
' Postconditions: Inspect the Immediate Window for the display modes,
' transparency states, and visibility states of all three components
' in both DS_1 and DS_2.
'-----

Option Explicit
Dim swApp As SldWorks.SldWorks
Dim swDoc As SldWorks.ModelDoc2
Dim swExt As SldWorks.ModelDocExtension
Dim swDSS As SldWorks.DisplayStateSetting
Dim varStatus As Variant
Dim varTStatus As Variant
Dim varVStatus As Variant
Dim statusArray(5) As Long
Dim statusTArray(5) As Long
Dim statusVArray(5) As Long
Dim varComp As Object
Dim varDS As Object
Dim returnCode As Boolean
Const maxEntMode As Integer = 3
Public Sub Main()
    Set swApp = Application.SldWorks
    Set swDoc = swApp.ActiveDoc
    Set swExt = swDoc.Extension
    Dim docType As Integer
    docType = swDoc.GetType
    If docType = swDocumentTypes_e.swDocASSEMBLY Then
        Set swDSS = swExt.GetDisplayStateSetting(swDisplayStateOpts_e.swThisDisplayState)
        swDSS.Option = swDisplayStateOpts_e.swSpecifyDisplayState
        Dim specDSNames(1) As String
        specDSNames(0) = "DS_1"
        specDSNames(1) = "DS_2"
        Dim varSpecDSNames As Variant
        varSpecDSNames = specDSNames
        swDSS.Names = varSpecDSNames
        Dim swADoc As AssemblyDoc
        Set swADoc = swDoc
        Dim compCnt As Integer
        compCnt = swADoc.GetComponentCount(True)
        Dim listComp(maxEntMode - 1) As Component2
        If compCnt >= maxEntMode Then
            Dim varComp As Variant
            varComp = swADoc.GetComponents(True)
            Set listComp(0) = varComp(0)
            Set listComp(1) = varComp(1)
            Set listComp(2) = varComp(2)
            swDSS.Entities = listComp
        End If

        varStatus = swExt.DisplayMode(swDSS)
        statusArray(0) = varStatus(0)
        statusArray(1) = varStatus(1)
        statusArray(2) = varStatus(2)
        statusArray(3) = varStatus(3)
        statusArray(4) = varStatus(4)
        statusArray(5) = varStatus(5)

        varTStatus = swExt.Transparency(swDSS)
        statusTArray(0) = varTStatus(0)
        statusTArray(1) = varTStatus(1)
        statusTArray(2) = varTStatus(2)
        statusTArray(3) = varTStatus(3)
        statusTArray(4) = varTStatus(4)
        statusTArray(5) = varTStatus(5)

        varVStatus = swExt.Visibility(swDSS)
        statusVArray(0) = varVStatus(0)
        statusVArray(1) = varVStatus(1)
        statusVArray(2) = varVStatus(2)
        statusVArray(3) = varVStatus(3)
        statusVArray(4) = varVStatus(4)
        statusVArray(5) = varVStatus(5)

        WriteOutput

    End If
End Sub

Public Sub WriteOutput()
    Dim entCount As Integer

```

```

entCount = swDSS.GetEntityCount
Dim listComp() As Component2
listComp = swDSS.Entities
Dim allCtr As Integer
allCtr = 0
Dim entctr As Long
For entctr = 0 To entCount - 1
    Dim swComp As Component2
    Set swComp = listComp(entctr)
    Debug.Print (swComp.Name2)
    Dim dsNameCount As Integer
    dsNameCount = swDSS.GetNameCount()
    Dim dsNames() As String
    dsNames = swDSS.Names
    Dim namectr As Long
    For namectr = 0 To dsNameCount - 1
        Debug.Print (" " & dsNames(namectr))
        Dim status As Integer
        status = statusArray(allCtr)
        Dim statusT As Integer
        statusT = statusTArray(allCtr)
        Dim statusV As Integer
        statusV = statusVArray(allCtr)
        WriteMode (status)
        WriteTransparency (statusT)
        WriteVisibility (statusV)
        allCtr = allCtr + 1
    Next
Next
End Sub
Public Sub WriteMode(ByVal status As Integer)
    Select Case status
        Case CInt(swDisplayMode_e.swDisplayModeDEFAULT)
            Debug.Print (" swDisplayModeDEFAULT")

        Case CInt(swDisplayMode_e.swHIDDEN)
            Debug.Print (" swHIDDEN")

        Case CInt(swDisplayMode_e.swHIDDEN_GREYED)
            Debug.Print (" swHIDDEN_GREYED")

        Case CInt(swDisplayMode_e.swSHADED)
            Debug.Print (" swSHADED")

        Case CInt(swDisplayMode_e.swSHADED_EDGES)
            Debug.Print (" swSHADED_EDGES")

        Case CInt(swDisplayMode_e.swWIREFRAME)
            Debug.Print (" swWIREFRAME")

        Case CInt(swDisplayMode_e.swDisplayModeUNKNOWN)
            Debug.Print (" Error:swDisplayModeUNKNOWN")

        Case CInt(swDisplayMode_e.swFACETED_WIREFRAME)
            Debug.Print (" swFACETED_WIREFRAME")

        Case CInt(swDisplayMode_e.swFACETED_HIDDEN_GREYED)
            Debug.Print (" swFACETED_HIDDEN_GREYED")

        Case CInt(swDisplayMode_e.swFACETED_HIDDEN)
            Debug.Print (" swFACETED_HIDDEN")

    End Select
End Sub
Public Sub WriteTransparency(ByVal status As Integer)
    Select Case status
        Case CInt(swTransparencyState_e.swTransparencyStateTransparent)
            Debug.Print (" swTransparencyStateTransparent")

        Case CInt(swTransparencyState_e.swTransparencyStateNonTransparent)
            Debug.Print (" swTransparencyStateNonTransparent")

        Case CInt(swTransparencyState_e.swTransparencyStateUnknown)
            Debug.Print (" ERROR : swTransparencyStateUnknown")

    End Select
End Sub
Public Sub WriteVisibility(ByVal status As Integer)
    Select Case status
        Case CInt(swVisibilityState_e.swVisibilityStateHide)
            Debug.Print (" swVisibilityStateHide")

        Case CInt(swVisibilityState_e.swVisibilityStateShown)
            Debug.Print (" swVisibilityStateShown")

        Case CInt(swVisibilityState_e.swVisibilityStateUnknown)
            Debug.Print (" ERROR : swVisibilityStateUnknown")

    End Select
End Sub

```


Get Display State Settings Example (VB.NET)

This example shows how to get display modes, transparency states, and visibility states of components.

```

'-----
' Preconditions: Open an assembly that contains a minimum of three top-level
' components and two display states, "DS_1" and "DS_2".
'-----
' Postconditions: Inspect the Immediate Window for the display modes,
' transparency states, and visibility states of all three components
' in both DS_1 and DS_2.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Diagnostics

Partial Public Class SolidWorksMacro
    Private swDoc As ModelDoc2 = Nothing
    Private swExt As ModelDocExtension = Nothing
    Private swDSS As DisplayStateSetting = Nothing
    Private varStatus As Object
    Private varTStatus As Object
    Private varVStatus As Object
    Private statusArray As Array
    Private statusTArray As Array
    Private statusVArray As Array
    Const maxEntMode As Integer = 3

    Public Sub Main()
        swDoc = DirectCast(swApp.ActiveDoc, ModelDoc2)
        swExt = swDoc.Extension
        Dim docType As Integer = swDoc.GetType]()
        If docType = CInt(swDocumentTypes_e.swDocASSEMBLY) Then

            CreateDisplayStateSetting()

            varStatus = swExt.DisplayMode(swDSS)
            statusArray = DirectCast(varStatus, Array)

            varTStatus = swExt.Transparency(swDSS)
            statusTArray = DirectCast(varTStatus, Array)

            varVStatus = swExt.Visibility(swDSS)
            statusVArray = DirectCast(varVStatus, Array)

            WriteOutput()

        End If
    End Sub

    Public Sub CreateDisplayStateSetting()
        swDSS = swExt.GetDisplayStateSetting(CInt(swDisplayStateOpts_e.swThisDisplayState))
        swDSS.Option = CInt(swDisplayStateOpts_e.swSpecifyDisplayState)

        Dim specDSNames As String() = New String(1) {}
        specDSNames(0) = "DS_1"
        specDSNames(1) = "DS_2"
        Dim varSpecDSNames As Object = specDSNames
        swDSS.Names = varSpecDSNames

        Dim swADoc As AssemblyDoc
        swADoc = DirectCast(swDoc, AssemblyDoc)
        Dim compCnt As Integer = swADoc.GetComponentCount(True)
        Dim listComp As Component2() = New Component2(maxEntMode - 1) {}
        If compCnt >= maxEntMode Then
            Dim varComp As Object() = DirectCast(swADoc.GetComponents(True), Object())
            listComp(0) = DirectCast(varComp(0), Component2)
            listComp(1) = DirectCast(varComp(1), Component2)
            listComp(2) = DirectCast(varComp(2), Component2)
            swDSS.Entities = listComp
        End If
    End Sub

    Public Sub WriteOutput()
        Dim entCount As Integer = swDSS.GetEntityCount()
        Dim listComp As Object() = DirectCast(swDSS.Entities, Object())
        Dim allCtr As Integer = 0
        For entCtr As Integer = 0 To entCount - 1
            Dim swComp As Component2 = DirectCast(listComp(entCtr), Component2)
            Debug.Print(swComp.Name2)
            Dim dsNameCount As Integer = swDSS.GetNameCount()
            Dim dsNames As Object() = DirectCast(swDSS.Names, Object())

            For nameCtr As Integer = 0 To dsNameCount - 1
                Debug.Print("  " & DirectCast(dsNames(nameCtr), String))
                Dim status As Integer = CInt(statusArray.GetValue(allCtr))
                Dim statusT As Integer = CInt(statusTArray.GetValue(allCtr))
                Dim statusV As Integer = CInt(statusVArray.GetValue(allCtr))
            End For
        End For
    End Sub
End Class

```

```

        WriteMode(status)
        WriteTransparency(statusT)
        WriteVisibility(statusV)
        allCtr += 1
    Next
Next
End Sub
Public Sub WriteMode(ByVal status As Integer)
    Select Case status
        Case CInt(swDisplayMode_e.swDisplayModeDEFAULT)
            Debug.Print("    swDisplayModeDEFAULT")
            Exit Select
        Case CInt(swDisplayMode_e.swHIDDEN)
            Debug.Print("    swHIDDEN")
            Exit Select
        Case CInt(swDisplayMode_e.swHIDDEN_GREYED)
            Debug.Print("    swHIDDEN_GREYED")
            Exit Select
        Case CInt(swDisplayMode_e.swSHADED)
            Debug.Print("    swSHADED")
            Exit Select
        Case CInt(swDisplayMode_e.swSHADED_EDGES)
            Debug.Print("    swSHADED_EDGES")
            Exit Select
        Case CInt(swDisplayMode_e.swWIREFRAME)
            Debug.Print("    swWIREFRAME")
            Exit Select
        Case CInt(swDisplayMode_e.swDisplayModeUNKNOWN)
            Debug.Print("    Error:swDisplayModeUNKNOWN")
            Exit Select
        Case CInt(swDisplayMode_e.swFACETED_WIREFRAME)
            Debug.Print("    swFACETED_WIREFRAME")
            Exit Select
        Case CInt(swDisplayMode_e.swFACETED_HIDDEN_GREYED)
            Debug.Print("    swFACETED_HIDDEN_GREYED")
            Exit Select
        Case CInt(swDisplayMode_e.swFACETED_HIDDEN)
            Debug.Print("    swFACETED_HIDDEN")
            Exit Select
    End Select
End Sub
Public Sub WriteTransparency(ByVal status As Integer)
    Select Case status
        Case CInt(swTransparencyState_e.swTransparencyStateTransparent)
            Debug.Print("    swTransparencyStateTransparent")
            Exit Select
        Case CInt(swTransparencyState_e.swTransparencyStateNonTransparent)
            Debug.Print("    swTransparencyStateNonTransparent")
            Exit Select
        Case CInt(swTransparencyState_e.swTransparencyStateUnknown)
            Debug.Print("    ERROR : swTransparencyStateUnknown")
            Exit Select
    End Select
End Sub
Public Sub WriteVisibility(ByVal status As Integer)
    Select Case status
        Case CInt(swVisibilityState_e.swVisibilityStateHide)
            Debug.Print("    swVisibilityStateHide")
            Exit Select
        Case CInt(swVisibilityState_e.swVisibilityStateShown)
            Debug.Print("    swVisibilityStateShown")
            Exit Select
        Case CInt(swVisibilityState_e.swVisibilityStateUnknown)
            Debug.Print("    ERROR : swVisibilityStateUnknown")
            Exit Select
    End Select
End Sub

Public swApp As SldWorks
End Class

```

Get Distance Between Entities Example (C#)

This example shows how to get the minimum and maximum distances between face and edge entities.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\multibody\multi_inter.sldprt.
// 2. Open the Immediate window.
// 3. Put your cursor anywhere in the Main module in the IDE and press F5.
//
// Postconditions:
// 1. Creates a sketch line that depicts the maximum distance between
//    the face and edge entities.
// 2. Put your cursor over Sketch4 in the FeatureManager design tree and
//    examine the graphics area.
// 3. Examine the Immediate window.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace GetDistanceBetweenEntities_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swDoc;
        SelectionMgr swSM;
        Face2 swFace;
        Edge swEdge;
        Entity swTop1;
        Entity swTop2;
        bool bMin;
        long retval;
        double dist;
        double[] varParam;
        object varPos1;
        object varPos2;
        double[] Pos1 = new double[3];
        double[] Pos2 = new double[3];
        int caseType;
        bool boolstatus;

        public void Main()
        {
            varParam = null;
            swDoc = (ModelDoc2)swApp.ActiveDoc;
            swSM = (SelectionMgr)swDoc.SelectionManager;

            for (caseType = 1; caseType <= 2; caseType++)
            {
                switch (caseType)
                {
                    case 1:
                        FaceFaceMinDistance();
                        break;
                    case 2:
                        FaceEdgeMaxDistance();
                        break;
                    default:
                        break;
                }
            }
        }

        public void SetParameterForEdge()
        {
            double[] startPt = new double[3]; ;
            double[] endPt = new double[3]; ;
            double[] startPara = new double[2]; ;
            double[] endPara = new double[2]; ;

            swEdge = (Edge)swSM.GetSelectedObject6(2, -1);
            startPt = (double[])((Vertex)swEdge.GetStartVertex()).GetPoint();
            endPt = (double[])((Vertex)swEdge.GetEndVertex()).GetPoint();
            startPara = (double[])swEdge.GetParameter(startPt[0], startPt[1], startPt[2]);
            endPara = (double[])swEdge.GetParameter(endPt[0], endPt[1], endPt[2]);

            double[] paramDl = new double[2];
            paramDl[0] = startPara[0];
            paramDl[1] = endPara[0];
            varParam = paramDl;
        }

        public void SetParameterForFace()
        {

```

```

{
    swFace = (Face2)swSM.GetSelectedObject6(2, -1);
    Surface swSurface = default(Surface);
    swSurface = (Surface)swFace.GetSurface();
    double[] varBox = new double[6];
    varBox = (double[])swFace.GetBox();
    double[] varLowParam = new double[2];
    double[] varHighParam = new double[2];
    varLowParam = (double[])swSurface.ReverseEvaluate(varBox[0], varBox[1], varBox[2]);
    varHighParam = (double[])swSurface.ReverseEvaluate(varBox[3], varBox[4], varBox[5]);

    double[] paramD2 = new double[4];
    paramD2[0] = varLowParam[0];
    paramD2[1] = varLowParam[1];
    paramD2[2] = varHighParam[0];
    paramD2[3] = varHighParam[1];
    varParam = paramD2;
}

public void CreateLine()
{
    SketchManager swSkM = default(SketchManager);
    SketchSegment skSegment = default(SketchSegment);
    swSkM = swDoc.SketchManager;
    swDoc.Extension.SelectByID2("Front", "PLANE", 0, 0, 0, false, 0, null, 0);
    swSkM.InsertSketch(true);
    skSegment = swSkM.CreateLine(Pos1[0], Pos1[1], Pos1[2], Pos2[0], Pos2[1], Pos2[2]);
    swDoc.ClearSelection2(true);
    swSkM.InsertSketch(true);
}

public void FaceFaceMinDistance()
{
    swDoc.ClearSelection();
    boolstatus = swDoc.Extension.SelectByID2("", "FACE", -0.07448317477082, -0.04390354307787, 0.08443247713632, false, 0, r
    boolstatus = swDoc.Extension.SelectByID2("", "FACE", -0.05640517674067, -0.005486808589922, 0.06500000000005, true, 0, r
    SetParameterForFace();
    if ((swSM.GetSelectedObjectCount() == 2))
    {
        swTop1 = (Entity)swSM.GetSelectedObject6(1, -1);
        swTop2 = (Entity)swSM.GetSelectedObject6(2, -1);
        bMin = true;
        retval = swTop1.GetDistance(swTop2, bMin, varParam, out varPos1, out varPos2, out dist);
        Pos1 = (double[])varPos1;
        Pos2 = (double[])varPos2;
        Debug.Print("IEquity::GetDistance return value (0 = success; 1 = failure) : " + retval);
        Debug.Print("Face1 coordinate: " + Pos1[0] + "," + Pos1[1] + "," + Pos1[2]);
        Debug.Print("Face2 coordinate: " + Pos2[0] + "," + Pos2[1] + "," + Pos2[2]);
        Debug.Print("Minimum distance between two faces = " + dist * 1000 + " mm");
        Debug.Print("");
        CreateLine();
    }
}

public void FaceEdgeMaxDistance()
{
    swDoc.ClearSelection();
    boolstatus = swDoc.Extension.SelectByID2("", "FACE", -0.0712080569869, -0.04996892464504, 0.08163440548356, false, 0, nu
    boolstatus = swDoc.Extension.SelectByID2("", "EDGE", -0.04898677039967, 0.0004196506486664, 0.06476403488529, true, 0, r
    SetParameterForEdge();
    if ((swSM.GetSelectedObjectCount() == 2))
    {
        swTop1 = (Entity)swSM.GetSelectedObject6(1, -1);
        swTop2 = (Entity)swSM.GetSelectedObject6(2, -1);
        bMin = false;
        retval = swTop1.GetDistance(swTop2, bMin, varParam, out varPos1, out varPos2, out dist);
        Pos1 = (double[])varPos1;
        Pos2 = (double[])varPos2;
        Debug.Print("IEquity::GetDistance return value (0 = success; 1 = failure) : " + retval);
        Debug.Print("Face coordinate: " + Pos1[0] + "," + Pos1[1] + "," + Pos1[2]);
        Debug.Print("Edge coordinate: " + Pos2[0] + "," + Pos2[1] + "," + Pos2[2]);
        Debug.Print("Maximum distance between face and edge = " + dist * 1000 + " mm");
        Debug.Print("");
        CreateLine();
    }
}

public SldWorks swApp;
}
}

```

Get Distance Between Entities Example (VBA)

This example shows how to get the minimum and maximum distances between face and edge entities.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\multibody\multi_inter.sldprt.
' 2. Open the Immediate window.
' 3. Put your cursor anywhere in the main module in the IDE and press F5.
'
' Postconditions:
' 1. Creates a sketch line that depicts the maximum distance between
'    the face and edge entities.
' 2. Put your cursor over Sketch4 in the FeatureManager design tree and
'    examine the graphics area.
' 3. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swDoc As SldWorks.ModelDoc2
Dim swSM As SldWorks.SelectionMgr
Dim swFace As SldWorks.Face2
Dim swEdge As SldWorks.Edge
Dim swTop1 As SldWorks.Entity
Dim swTop2 As SldWorks.Entity
Dim bMin As Boolean
Dim retVal As Long
Dim dist As Double
Dim varParam As Variant
Dim varPos1 As Variant
Dim varPos2 As Variant
Dim caseType As Integer
Dim boolstatus As Boolean

Sub main()
    varParam = Empty

    Set swApp = Application.SldWorks
    Set swDoc = swApp.ActiveDoc
    Set swSM = swDoc.SelectionManager

    For caseType = 1 To 2
        Select Case caseType
            Case 1
                FaceFaceMinDistance
            Case 2
                FaceEdgeMaxDistance
            Case Else
                Exit Sub
        End Select
    Next

    Set swTop1 = Nothing
    Set swTop2 = Nothing
    Set swFace = Nothing
    Set swEdge = Nothing
    Set swSM = Nothing
    Set swDoc = Nothing
    Set swApp = Nothing

End Sub

Sub SetParameterForEdge()
    Dim startPt As Variant
    Dim startVertex As Variant
    Dim endPt As Variant
    Dim endVertex As Variant
    Dim startPara As Variant
    Dim endPara As Variant

    Set swEdge = swSM.GetSelectedObject6(2, -1)
    Set startVertex = swEdge.GetStartVertex
    startPt = startVertex.GetPoint
    endPt = swEdge.GetEndVertex.GetPoint
    startPara = swEdge.GetParameter(startPt(0), startPt(1), startPt(2))
    endPara = swEdge.GetParameter(endPt(0), endPt(1), endPt(2))

    Dim paramDl(1) As Double
    paramDl(0) = startPara(0)
    paramDl(1) = endPara(0)
    varParam = paramDl

End Sub

Sub SetParameterForFace()
    Set swFace = swSM.GetSelectedObject6(2, -1)
    Dim swSurface As SldWorks.Surface
    Set swSurface = swFace.GetSurface
    Dim varBox As Variant
    varBox = swFace.GetBox
    Dim varLowParam As Variant, varHighParam As Variant
    varLowParam = swSurface.ReverseEvaluate(varBox(0), varBox(1), varBox(2))
    varHighParam = swSurface.ReverseEvaluate(varBox(3), varBox(4), varBox(5))

    Dim paramD2(3) As Double
    paramD2(0) = varLowParam(0)
```

```

paramD2(1) = varLowParam(1)
paramD2(2) = varHighParam(0)
paramD2(3) = varHighParam(1)
varParam = paramD2
End Sub

Sub CreateLine()
Dim swSkM As SldWorks.SketchManager
Dim skSegment As SldWorks.SketchSegment
Set swSkM = swDoc.SketchManager
swDoc.Extension.SelectByID "Front", "PLANE", 0, 0, 0, False, 0, Nothing, 0
swSkM.InsertSketch True
Set skSegment = swSkM.CreateLine(varPos1(0), varPos1(1), varPos1(2), varPos2(0), varPos2(1), varPos2(2))
swDoc.ClearSelection2 True
swSkM.InsertSketch True
End Sub

Sub FaceFaceMinDistance()
swDoc.ClearSelection
boolstatus = swDoc.Extension.SelectByID("", "FACE", -0.07448317477082, -0.04390354307787, 0.08443247713632, False, 0, Nothing, 0)
boolstatus = swDoc.Extension.SelectByID("", "FACE", -0.05640517674067, -0.005486808589922, 0.065000000000005, True, 0, Nothing, 0)
SetParameterForFace
If (swSM.GetSelectedObjectCount = 2) Then
Set swTop1 = swSM.GetSelectedObject6(1, -1)
Set swTop2 = swSM.GetSelectedObject6(2, -1)
bMin = True
retval = swTop1.GetDistance(swTop2, bMin, varParam, varPos1, varPos2, dist)
Debug.Print ("IEntity::GetDistance return value (0 = success; 1 = failure): " & retval)
Debug.Print ("Face1 coordinate: " & varPos1(0) & ", " & varPos1(1) & ", " & varPos1(2))
Debug.Print ("Face2 coordinate: " & varPos2(0) & ", " & varPos2(1) & ", " & varPos2(2))
Debug.Print ("Minimum distance between two faces = " & dist * 1000 & " mm")
CreateLine
End If
End Sub

Sub FaceEdgeMaxDistance()
swDoc.ClearSelection
boolstatus = swDoc.Extension.SelectByID("", "FACE", -0.0712080569869, -0.04996892464504, 0.08163440548356, False, 0, Nothing, 0)
boolstatus = swDoc.Extension.SelectByID("", "EDGE", -0.04898677039967, 4.196506486664E-04, 0.06476403488529, True, 0, Nothing, 0)
SetParameterForEdge
If (swSM.GetSelectedObjectCount = 2) Then
Set swTop1 = swSM.GetSelectedObject6(1, -1)
Set swTop2 = swSM.GetSelectedObject6(2, -1)
bMin = False
retval = swTop1.GetDistance(swTop2, bMin, varParam, varPos1, varPos2, dist)
Debug.Print ("IEntity::GetDistance return value (0 = success; 1 = failure): " & retval)
Debug.Print ("Face coordinate: " & varPos1(0) & ", " & varPos1(1) & ", " & varPos1(2))
Debug.Print ("Edge coordinate: " & varPos2(0) & ", " & varPos2(1) & ", " & varPos2(2))
Debug.Print ("Maximum distance between face and edge = " & dist * 1000 & " mm")
CreateLine
End If
End Sub

```

Get Distance Between Entities Example (VB.NET)

This example shows how to get the minimum and maximum distances between face and edge entities.

```

Preconditions:
1. Open public.documents\samples\tutorial\multibody\multi_inter.sldprt.
2. Open the Immediate window.
3. Put your cursor anywhere in the main module in the IDE and press F5.

Postconditions:
1. Creates a sketch line that depicts the maximum distance between
   the face and edge entities.
2. Put your cursor over Sketch4 in the FeatureManager design tree and
   examine the graphics area.
3. Examine the Immediate window.

NOTE: Because the part is used elsewhere, do not save changes.

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```

    Dim swDoc As ModelDoc2
    Dim swSM As SelectionMgr
    Dim swFace As Face2
    Dim swEdge As Edge
    Dim swTop1 As Entity
    Dim swTop2 As Entity
    Dim bMin As Boolean
    Dim retval As Long
    Dim dist As Double
    Dim varParam As Object
    Dim varPos1 As Object
    Dim varPos2 As Object
    Dim caseType As Integer
    Dim boolstatus As Boolean

    Sub main()

        varParam = Nothing
        swDoc = swApp.ActiveDoc
        swSM = swDoc.SelectionManager

        For caseType = 1 To 2
            Select Case caseType
                Case 1
                    FaceFaceMinDistance()
                Case 2
                    FaceEdgeMaxDistance()
                Case Else
                    Exit Sub
            End Select
        Next

        swTop1 = Nothing
        swTop2 = Nothing
        swFace = Nothing
        swEdge = Nothing
        swSM = Nothing
        swDoc = Nothing
        swApp = Nothing
    End Sub

    Sub SetParameterForEdge()
        Dim startPt As Object
        Dim startVertex As Object
        Dim endPt As Object
        Dim startPara As Object
        Dim endPara As Object

        swEdge = swSM.GetSelectedObject6(2, -1)
        startVertex = swEdge.GetStartVertex
        startPt = startVertex.GetPoint
        endPt = swEdge.GetEndVertex.GetPoint
        startPara = swEdge.GetParameter(startPt(0), startPt(1), startPt(2))
        endPara = swEdge.GetParameter(endPt(0), endPt(1), endPt(2))

        Dim paramDl(1) As Double
        paramDl(0) = startPara(0)
        paramDl(1) = endPara(0)
        varParam = paramDl
    End Sub

    Sub SetParameterForFace()
        swFace = swSM.GetSelectedObject6(2, -1)
        Dim swSurface As Surface
        swSurface = swFace.GetSurface
        Dim varBox As Object
        varBox = swFace.GetBox
        Dim varLowParam As Object, varHighParam As Object
        varLowParam = swSurface.ReverseEvaluate(varBox(0), varBox(1), varBox(2))
        varHighParam = swSurface.ReverseEvaluate(varBox(3), varBox(4), varBox(5))

        Dim paramD2(3) As Double
        paramD2(0) = varLowParam(0)
        paramD2(1) = varLowParam(1)
        paramD2(2) = varHighParam(0)
        paramD2(3) = varHighParam(1)
        varParam = paramD2
    End Sub

    Sub CreateLine()
        Dim swSkM As SketchManager
        Dim skSegment As SketchSegment
        swSkM = swDoc.SketchManager
        swDoc.Extension.SelectByID2("Front", "PLANE", 0, 0, 0, False, 0, Nothing, 0)

```

```

swSkM.InsertSketch(True)
skSegment = swSkM.CreateLine(varPos1(0), varPos1(1), varPos1(2), varPos2(0), varPos2(1), varPos2(2))
swDoc.ClearSelection2(True)
swSkM.InsertSketch(True)
End Sub

Sub FaceFaceMinDistance()
swDoc.ClearSelection()
boolstatus = swDoc.Extension.SelectByID2("", "FACE", -0.07448317477082, -0.04390354307787, 0.08443247713632, False, 0, Nothing, 0)
boolstatus = swDoc.Extension.SelectByID2("", "FACE", -0.05640517674067, -0.005486808589922, 0.065000000000005, True, 0, Nothing, 0)
SetParameterForFace()
If (swSM.GetSelectedObjectCount = 2) Then
swTop1 = swSM.GetSelectedObject6(1, -1)
swTop2 = swSM.GetSelectedObject6(2, -1)
bMin = True
retval = swTop1.GetDistance(swTop2, bMin, varParam, varPos1, varPos2, dist)
Debug.Print("IEntity::GetDistance return value (0 = success; 1 = failure) : " & retval)
Debug.Print("Face1 coordinate: " & varPos1(0) & "," & varPos1(1) & "," & varPos1(2))
Debug.Print("Face2 coordinate: " & varPos2(0) & "," & varPos2(1) & "," & varPos2(2))
Debug.Print("Minimum distance between two faces = " & dist * 1000 & " mm")
Debug.Print("")
CreateLine()
End If
End Sub

Sub FaceEdgeMaxDistance()
swDoc.ClearSelection()
boolstatus = swDoc.Extension.SelectByID2("", "FACE", -0.0712080569869, -0.04996892464504, 0.08163440548356, False, 0, Nothing, 0)
boolstatus = swDoc.Extension.SelectByID2("", "EDGE", -0.04898677039967, 0.0004196506486664, 0.06476403488529, True, 0, Nothing, 0)
SetParameterForEdge()
If (swSM.GetSelectedObjectCount = 2) Then
swTop1 = swSM.GetSelectedObject6(1, -1)
swTop2 = swSM.GetSelectedObject6(2, -1)
bMin = False
retval = swTop1.GetDistance(swTop2, bMin, varParam, varPos1, varPos2, dist)
Debug.Print("IEntity::GetDistance return value (0 = success; 1 = failure) : " & retval)
Debug.Print("Face coordinate: " & varPos1(0) & "," & varPos1(1) & "," & varPos1(2))
Debug.Print("Edge coordinate: " & varPos2(0) & "," & varPos2(1) & "," & varPos2(2))
Debug.Print("Maximum distance between face and edge = " & dist * 1000 & " mm")
Debug.Print("")
CreateLine()
End If
End Sub

Public swApp As SldWorks

End Class

```

Get Document Information Example (VBA)

This example shows how to get the title and document type of the active document.

```
'-----
Sub ModelInfo( )

    ' Define variable used to hold the SldWorks object
    Dim swApp As Object

    ' Define variable used to hold the ModelDoc object
    Dim Model As Object
    Dim TopName As String

    ' Constant enumerators
    Const swDocPART = 1
    Const swDocASSEMBLY = 2
    Const swDocDRAWING = 3

    ' Create or attach to SOLIDWORKS Design session. Because you are grabbing the active document, this example requires that SOLIDWORKS Design be running and have a document already loaded.
    Set swApp = CreateObject("SldWorks.Application")

    ' Grab the currently active document
    Set Model = swApp.ActiveDoc

    ' Check to see if a document is loaded
    If Model Is Nothing Then
        ' If no model is currently loaded, then exit
        Exit Sub
    End If

    ' Determine the document type. If the document is a drawing, then send a message to the user.
    If (Model.GetType <> swDocDRAWING) Then
        swApp.SendMsgToUser ("This is not a drawing")
    Else
        ' Get the document title
        Let TopName = Model.GetTitle
    End If

End Sub
```

Get Document Referenced by Drawing View Example (VBA)

This example shows how to get the name of the document referenced by a drawing view.

```
'-----  
' Preconditions:  
' 1. Open a drawing document.  
' 2. Select a drawing view.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets the name of the document referenced in the  
'    selected drawing view.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swView As SldWorks.View  
    Dim swDrawModel As SldWorks.ModelDoc2  
    Dim sModelName As String  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swView = swSelMgr.GetSelectedObject6(1, -1)  
    Set swDrawModel = swView.ReferencedDocument  
    sModelName = swView.GetReferencedModelName  
    Debug.Print "File" = " & swModel.GetPathName  
    Debug.Print " View" = " & swView.Name  
    Debug.Print " Referenced model name = " & sModelName  
    Debug.Print " Model path" = " & swDrawModel.GetPathName  
  
End Sub
```

Get Drawing Template Size Example (VBA)

This example shows how to get the size of the specified drawing template.

```
'-----  
Option Explicit  
Public Enum swDwgPaperSizes_e  
    swDwgPaperASize = 0  
    swDwgPaperASizeVertical = 1  
    swDwgPaperBSize = 2  
    swDwgPaperCSize = 3  
    swDwgPaperDSize = 4  
    swDwgPaperEsize = 5  
    swDwgPaperA4size = 6  
    swDwgPaperA4sizeVertical = 7  
    swDwgPaperA3size = 8  
    swDwgPaperA2size = 9  
    swDwgPaperA1size = 10  
    swDwgPaperA0size = 11  
    swDwgPapersUserDefined = 12  
End Enum  
Sub main()  
    Const sTemplateName As String = "c:\Program Files\SOLIDWORKS\data\templates\drawing.drwdot"  
  
    Dim swApp           As SldWorks.SldWorks  
    Dim vTemplateSizes  As Variant  
    Set swApp = CreateObject("SldWorks.Application")  
    vTemplateSizes = swApp.GetTemplateSizes(sTemplateName)  
  
    Debug.Print "Template = " & sTemplateName  
    Debug.Print " Paper Size  = " & vTemplateSizes(0)  
    Debug.Print " Width    = " & vTemplateSizes(1) * 1000# & " mm"  
    Debug.Print " Height   = " & vTemplateSizes(2) * 1000# & " mm"  
End Sub  
'-----
```

Get Drawing View Names and Types Example (C#)

This example shows how to get the names and types of all of the drawing views on the current sheet.

```
//-----
// Preconditions:
// 1. Open a drawing document.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Traverses the views on the current sheet and gets each view's type.
// 2. Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace GetNextView_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            {
                ModelDoc2 swModel = default(ModelDoc2);
                DrawingDoc swDraw = default(DrawingDoc);
                Sheet swSheet = default(Sheet);
                View swView = default(View);

                swModel = (ModelDoc2)swApp.ActiveDoc;
                swDraw = (DrawingDoc)swModel;
                swSheet = (Sheet)swDraw.GetCurrentSheet();
                swView = (View)swDraw.GetFirstView();

                Debug.Print("File = " + swModel.GetPathName());
                Debug.Print(" " + swSheet.GetName());

                while ((swView != null))
                {
                    Debug.Print(" " + swView.GetName2() + " [" + swView.Type + "]");
                    swView = (View)swView.GetNextView();
                }
            }

            public SldWorks swApp;
        }
    }
}
```

Get Drawing View Names and Types Example (VBA)

This example shows how to get the names and types of all of the drawing views on the current sheet.

```
'-----  
' Preconditions:  
' 1. Open a drawing document.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Traverses the views on the current sheet and gets each view's type.  
' 2. Inspect the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swDraw As SldWorks.DrawingDoc  
    Dim swSheet As SldWorks.Sheet  
    Dim swView As SldWorks.View  
    Dim bRet As Boolean  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set swDraw = swModel  
    Set swSheet = swDraw.GetCurrentSheet  
    Set swView = swDraw.GetFirstView  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print " " & swSheet.GetName  
  
    While Not swView Is Nothing  
        Debug.Print " " & swView.GetName2 & " [" & swView.Type & "]"  
        Set swView = swView.GetNextView  
    Wend  
  
End Sub
```

Get Drawing View Names and Types Example (VB.NET)

This example shows how to get the names and types of all of the drawing views on the current sheet.

```
'-----  
' Preconditions:  
' 1. Open a drawing document.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Traverses the views on the current sheet and gets each view's type.  
' 2. Inspect the Immediate window.  
'-----  
  
Imports SolidWorks.Interop.sldworks  
  
Imports SolidWorks.Interop.swconst  
  
Imports System.Runtime.InteropServices  
  
Imports System  
  
Imports System.Diagnostics  
  
Partial Class SolidWorksMacro  
  
    Sub main()  
  
        Dim swModel As ModelDoc2  
  
        Dim swDraw As DrawingDoc  
  
        Dim swSheet As Sheet  
  
        Dim swView As View  
  
        swModel = swApp.ActiveDoc  
  
        swDraw = swModel  
  
        swSheet = swDraw.GetCurrentSheet  
  
        swView = swDraw.GetFirstView  
  
        Debug.Print("File = " & swModel.GetPathName)  
  
        Debug.Print(" " & swSheet.GetName)  
  
        While Not swView Is Nothing  
            Debug.Print(" " & swView.GetName2 & " [" & swView.Type & "]")  
  
            swView = swView.GetNextView  
  
        End While  
  
    End Sub  
  
    Public swApp As SldWorks  
  
End Class
```

Get DXF User Preference Settings Example (VBA)

This example shows how to get DXF user-preference settings.

```
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim DxfVersion As Integer  
Dim DxfOutputFonts As Integer  
Dim DxfMappingFileIndex As Integer  
Dim AutoSaveInterval As Integer  
  
Sub mian()  
  
Set swApp = Application.SldWorks  
  
' Get the version of DXF for exporting  
DxfVersion = swApp.GetUserPreferenceIntegerValue(swDxfVersion)  
  
' Get whether TrueType fonts are exported (1) or only standard (0)  
DxfOutputFonts = swApp.GetUserPreferenceIntegerValue(swDxfOutputFonts)  
Debug.Print "Output fonts: exported =1; standard = 0: "; DxfOutputFonts  
  
' Get the index of the map file for the file used for custom DXF mapping  
DxfMappingFileIndex = swApp.GetUserPreferenceIntegerValue(swDxfMappingFileIndex)  
Debug.Print "Mapping file index: "; DxfMappingFileIndex  
  
' Get the number of operations between auto-saves  
AutoSaveInterval = swApp.GetUserPreferenceIntegerValue(swAutoSaveInterval)  
Debug.Print "Number of autosaves: "; AutoSaveInterval  
End Sub
```

Get Each Spline's Parameters Example (C#)

This example shows how to get each spline's parameters in a sketch containing multiple splines.

```
//-----
// Preconditions:
// 1. Verify that the specified part template exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Creates a sketch containing two splines.
// 2. Gets each spline's dimension, order, periodicity,
//    control point, and knot point data.
// 3. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            SketchSegment swSketchSegment = default(SketchSegment);
            Feature swFeature = default(Feature);
            Sketch swSketch = default(Sketch);
            SplineParamData swSplineParamData = default(SplineParamData);
            SketchManager swSketchMgr = default(SketchManager);
            double[] points = new double[12];
            object pointArray = null;
            object varCtrlPoints = null;
            double[] ctrlPoints;
            object varKnotPoints = null;
            double[] knotPoints;
            bool status = false;
            int i = 0;
            int j = 0;
            int k = 0;
            int splineCount = 0;
            int splinePointCount = 0;

            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2017\\templates\\Part.prtdot", 0, 0, 0);

            //Create a sketch with two splines
            //First spline
            points[0] = -0.185955019567672;
            points[1] = 0.0416208582005027;
            points[2] = 0;
            points[3] = -0.0862492383138544;
            points[4] = 0.0403922105323034;
            points[5] = 0;
            points[6] = -0.0672740896322921;
            points[7] = 0.0540598971292923;
            points[8] = 0;
            points[9] = -0.0141436733240425;
            points[10] = -0.00570437188125084;
            points[11] = 0;
            pointArray = points;
            swSketchMgr = swModel.SketchManager;
            swSketchSegment = swSketchMgr.CreateSpline((pointArray));
            swModel.ClearSelection2(true);
            //Second spline
            points[0] = -0.0838342193907238;
            points[1] = -0.0380341664350112;
            points[2] = 0;
            points[3] = -0.0655490761158148;
            points[4] = -0.0179490921124739;
            points[5] = 0;
            points[6] = -0.0179387030603664;
            points[7] = -0.0681344637902441;
            points[8] = 0;
            points[9] = 0.0634819349185705;
            points[10] = -0.0329692207162395;
            points[11] = 0;
            pointArray = points;
            swSketchSegment = (SketchSegment)swSketchMgr.CreateSpline((pointArray));
            swModel.ClearSelection2(true);
            //Sketch
            swSketchMgr.InsertSketch(true);

            //Get each spline's dimension, order, periodicity, control point, and knot data
            status = swModel.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, false, 0, null, (int)swSelectOption_e.swSelectOptionDefault);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swSketch = (Sketch)swFeature.GetSpecificFeature2();
            Debug.Print(swFeature.Name);
            Debug.Print("");
            splineCount = swSketch.GetSplineCount(ref splinePointCount);
            for (i = 1; i <= splineCount; i++)
            {
                Debug.Print("Spline " + i + ": ");
                swSplineParamData = (SplineParamData)swSketch.GetSplineParams5(true, i - 1);
                Debug.Print(" Dimension: " + swSplineParamData.Dimension);
                Debug.Print(" Order: " + swSplineParamData.Order);
                Debug.Print(" Periodicity: " + swSplineParamData.Periodic);
                Debug.Print(" Number of control points: " + swSplineParamData.ControlPointsCount);
                status = swSplineParamData.GetControlPoints(out varCtrlPoints);
                Debug.Print(" Control points:");
                ctrlPoints = (double[])varCtrlPoints;
                for (j = 0; j <= ctrlPoints.GetUpperBound(0); j++)
                {
                    Debug.Print(" " + ctrlPoints[j].ToString());
                }
                Debug.Print(" Number of knots: " + swSplineParamData.KnotPointsCount);
            }
        }
    }
}
```

```

        status = swSplineParamData.GetKnotPoints(out varKnotPoints);
        Debug.Print("    Knot points:");
        knotPoints = (double[])varKnotPoints;
        for (k = 0; k <= knotPoints.GetUpperBound(0); k++)
        {
            Debug.Print("        " + knotPoints[k].ToString());
        }
    }

}

/// <summary>
/// The Sldworks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Get Each Spline's Parameters Example (VBA)

This example shows how to get each spline's parameters in a sketch containing multiple splines.

```
'-----
' Preconditions:
' 1. Verify that the specified part template exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a sketch containing two splines.
' 2. Gets each spline's dimension, order, periodicity,
'    control point, and knot point data.
' 3. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swSketchSegment As SldWorks.SketchSegment
Dim swFeature As SldWorks.Feature
Dim swSketch As SldWorks.Sketch
Dim swSplineParamData As SldWorks.SplineParamData
Dim swSketchMgr As SldWorks.SketchManager
Dim points(11) As Double
Dim pointArray As Variant
Dim varCtrlPoints As Variant
Dim varKnotPoints As Variant
Dim status As Boolean
Dim i As Integer
Dim j As Integer
Dim k As Integer
Dim splineCount As Long
Dim splinePointCount As Long

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2017\templates\Part.prtdot", 0, 0, 0)

    'Create a sketch with two splines
    'First spline
    points(0) = -0.185955019567672
    points(1) = 4.16208582005027E-02
    points(2) = 0
    points(3) = -8.62492383138544E-02
    points(4) = 4.03922105323034E-02
    points(5) = 0
    points(6) = -6.72740896322921E-02
    points(7) = 5.40598971292923E-02
    points(8) = 0
    points(9) = -1.41436733240425E-02
    points(10) = -5.70437188125084E-03
    points(11) = 0
    pointArray = points
    Set swSketchMgr = swModel.SketchManager
    Set swSketchSegment = swSketchMgr.CreateSpline((pointArray))
    swModel.ClearSelection2 True
    'Second spline
    points(0) = -8.38342193907238E-02
    points(1) = -3.80341664350112E-02
    points(2) = 0
    points(3) = -6.55490761158148E-02
    points(4) = -1.79490921124739E-02
    points(5) = 0
    points(6) = -1.79387030603664E-02
    points(7) = -6.81344637902441E-02
    points(8) = 0
    points(9) = 6.34819349185705E-02
    points(10) = -3.29692207162395E-02
    points(11) = 0
    pointArray = points
    Set swSketchSegment = swSketchMgr.CreateSpline((pointArray))
    swModel.ClearSelection2 True
    'Sketch
    swSketchMgr.InsertSketch (True)

    'Get each spline's dimension, order, periodicity, control point, and knot data
    status = swModel.Extension.SelectByID("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, swSelectOptionDefault)
    Set swSelMgr = swModel.SelectionManager
    Set swFeature = swSelMgr.GetSelectedObject6(1, -1)
    Set swSketch = swFeature.GetSpecificFeature2
    Debug.Print swFeature.Name
    Debug.Print ""

    splineCount = swSketch.GetSplineCount(splinePointCount)

    For i = 1 To splineCount
        Debug.Print "Spline " & i & ": "
        Set swSplineParamData = swSketch.GetSplineParams5(True, i - 1)
        Debug.Print "    Dimension: " & swSplineParamData.Dimension
        Debug.Print "    Order: " & swSplineParamData.Order
        Debug.Print "    Periodicity: " & swSplineParamData.Periodic
        Debug.Print "    Number of control points: " & swSplineParamData.ControlPointsCount
```

```
status = swSplineParamData.GetControlPoints(varCtrlPoints)
Debug.Print "    Control points:"

For j = 0 To UBound(varCtrlPoints)
    Debug.Print "        " & varCtrlPoints(j)
Next j

Debug.Print "    Number of knots: " & swSplineParamData.KnotPointsCount
status = swSplineParamData.GetKnotPoints(varKnotPoints)
Debug.Print "        Knot points:"

For k = 0 To UBound(varKnotPoints)
    Debug.Print "            " & varKnotPoints(k)
Next k

Next i

End Sub
```

Get Each Spline's Parameters Example (VB.NET)

This example shows how to get each spline's parameters in a sketch containing multiple splines.

```

-----
Preconditions:
1. Verify that the specified part template exists.
2. Open the Immediate window.
Postconditions:
1. Creates a sketch containing two splines.
2. Gets each spline's dimension, order, periodicity,
   control point, and knot point data.
3. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swSketchSegment As SketchSegment
        Dim swFeature As Feature
        Dim swSketch As Sketch
        Dim swSplineParamData As SplineParamData
        Dim swSketchMgr As SketchManager
        Dim points(11) As Double
        Dim pointArray As Object
        Dim varCtrlPoints As Object = Nothing
        Dim varKnotPoints As Object = Nothing
        Dim status As Boolean
        Dim i As Integer
        Dim j As Integer
        Dim k As Integer
        Dim splineCount As Integer
        Dim splinePointCount As Integer

        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2017\templates\Part.prt", 0, 0, 0)

        'Create a sketch with two splines
        'First spline
        points(0) = -0.185955019567672
        points(1) = 0.0416208582005027
        points(2) = 0
        points(3) = -0.0862492383138544
        points(4) = 0.0403922105323034
        points(5) = 0
        points(6) = -0.0672740896322921
        points(7) = 0.0540598971292923
        points(8) = 0
        points(9) = -0.0141436733240425
        points(10) = -0.00570437188125084
        points(11) = 0
        pointArray = points
        swSketchMgr = swModel.SketchManager
        swSketchSegment = swSketchMgr.CreateSpline((pointArray))
        swModel.ClearSelection2(True)
        'Second spline
        points(0) = 0.0838342193907238
        points(1) = -0.0380341664350112
        points(2) = 0
        points(3) = -0.0655490761158148
        points(4) = -0.0179490921124739
        points(5) = 0
        points(6) = -0.0179387030603664
        points(7) = -0.0681344637902441
        points(8) = 0
        points(9) = 0.0634819349185705
        points(10) = -0.0329692207162395
        points(11) = 0
        pointArray = points
        swSketchSegment = swSketchMgr.CreateSpline((pointArray))
        swModel.ClearSelection2(True)
        'Sketch
        swSketchMgr.InsertSketch(True)

        'Get each spline's dimension, order, periodicity, control point, and knot data
        status = swModel.Extension.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, swSelectOption_e.swSelectOptionDefault)
        swSelMgr = swModel.SelectionManager
        swFeature = swSelMgr.GetSelectedObject6(1, -1)
        swSketch = swFeature.GetSpecificFeature2
        Debug.Print(swFeature.Name)
        Debug.Print("")
        splineCount = swSketch.GetSplineCount(splinePointCount)
        For i = 1 To splineCount
            Debug.Print("Spline " & i & ": ")
            swSplineParamData = swSketch.GetSplineParams5(True, i - 1)
            Debug.Print(" Dimension: " & swSplineParamData.Dimension)
            Debug.Print(" Order: " & swSplineParamData.Order)
            Debug.Print(" Periodicity: " & swSplineParamData.Periodic)
            Debug.Print(" Number of control points: " & swSplineParamData.ControlPointsCount)
            status = swSplineParamData.GetControlPoints(varCtrlPoints)
            Debug.Print(" Control points:")
            For j = 0 To UBound(varCtrlPoints)
                Debug.Print(" " & varCtrlPoints(j))
            Next j
            Debug.Print(" Number of knots: " & swSplineParamData.KnotPointsCount)
            status = swSplineParamData.GetKnotPoints(varKnotPoints)
            Debug.Print(" Knot points:")
            For k = 0 To UBound(varKnotPoints)
                Debug.Print(" " & varKnotPoints(k))
            Next k
        Next i
    End Sub
End Class

```

End Sub

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

End Class

Get Edge Chamfer Distances Example (C#)

This example shows how to get the distances of the edges and vertices for the chamfer feature.

```
//-----
// Preconditions:
// 1. Verify that the part document to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part document.
// 2. Creates a chamfer feature.
// 3. Examine the Immediate window for the chamfer data.
//
// NOTE: Because this part document is used elsewhere, do not
// save any changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace IChamferFeatureDataCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            //1 radian = 180°/p = 57.295779513° or approximately 57.3°
            const double DegPerRad = 57.3;
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Feature swFeat = default(Feature);
            FeatureManager swFeatMgr = default(FeatureManager);
            SelectionMgr swSelMgr = default(SelectionMgr);
            ChamferFeatureData2 swChamfer = default(ChamferFeatureData2);
            object[] vEdgeArr;
            object vEdge = null;
            Edge swEdge = default(Edge);
            Entity swEnt = default(Entity);
            SelectData swSelData = default(SelectData);
            int i = 0;
            bool bRet = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\block20.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Create and select chamfer feature
            bRet = swModelDocExt.SelectByID2("", "EDGE", -0.0621980171204655, 0.0394066118978458, -0.000546079442074188, true, 0, null, 0);
            bRet = swModelDocExt.SelectByID2("", "EDGE", -0.0381606508724985, 0.0396345009388028, 0.0493384681956286, true, 0, null, 1);
            swFeatMgr = (FeatureManager)swModel.FeatureManager;
            swFeat = (Feature)swFeatMgr.InsertFeatureChamfer(4, 1, 0.00254, 0.78539816339745, 0, 0, 0, 0);
            bRet = swModelDocExt.SelectByID2("Chamfer1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swSelData = (SelectData)swSelMgr.CreateSelectData();
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swChamfer = (ChamferFeatureData2)swFeat.GetDefinition();

            //Get chamfer information
            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print(" " + swFeat.Name);
            Debug.Print(" EdgeChamferAngle = " + swChamfer.EdgeChamferAngle * DegPerRad + " degrees");
            Debug.Print(" EqualDistance = " + swChamfer.EqualDistance);
            Debug.Print(" EdgeChamferDistance(0) = " + swChamfer.GetEdgeChamferDistance(0) * 1000.0 + " mm");
            Debug.Print(" EdgeChamferDistance(1) = " + swChamfer.GetEdgeChamferDistance(1) * 1000.0 + " mm");
            Debug.Print(" VertexChamferDistance(0) = " + swChamfer.GetVertexChamferDistance(0) * 1000.0 + " mm");
            Debug.Print(" VertexChamferDistance(1) = " + swChamfer.GetVertexChamferDistance(1) * 1000.0 + " mm");
            Debug.Print(" VertexChamferDistance(2) = " + swChamfer.GetVertexChamferDistance(2) * 1000.0 + " mm");
            Debug.Print(" KeepFeatures = " + swChamfer.KeepFeatures);
            Debug.Print(" GetFaceCount = " + swChamfer.GetFaceCount());
            Debug.Print(" GetEdgeCount = " + swChamfer.GetEdgeCount());
            Debug.Print(" Type = " + swChamfer.Type);
            // ChamferFeatureData2::Type
            // 1 = Angle-Distance
            // 2 = Distance-Distance
            // 3 = Vertex
            //Roll back to get access to geometric entities
            bRet = swChamfer.AccessSelections(swModel, null);
            Debug.Assert(bRet);
            vEdgeArr = (object[])swChamfer.Edges;

            if ((vEdgeArr != null))
            {
                swModel.ClearSelection2(true);
                i = 0;
                bRet = false;
                foreach (object vEdge_loopVariable in vEdgeArr)
                {
                    vEdge = (object)vEdge_loopVariable;
                    swEdge = (Edge)vEdge;
                    swEnt = (Entity)swEdge;
                    bRet = swEnt.Select4(true, swSelData);
                    Debug.Assert(bRet);
                    Debug.Print(" EdgeFlip(" + i + ") = " + swChamfer.GetIsFlipped(swEdge));
                    i = i + 1;
                }
            }

            //Roll forward
            swChamfer.ReleaseSelectionAccess();
        }
    }
}

/// <summary>
```

```
    /// The Sldworks swApp variable is pre-assigned for you.  
    /// </summary>  
    public SldWorks swApp;  
}
```

Get Edge Chamfer Distances Example (VBA)

This example shows how to get the distances of the edges and vertices for the chamfer feature.

```

'-----
' Preconditions:
' 1. Verify that the part document to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Creates a chamfer feature.
' 3. Examine the Immediate window for the chamfer data.
'
' NOTE: Because this part document is used elsewhere, do not
' save any changes.
'-----
Option Explicit

Sub main()

    ' 1 radian = 180°/p = 57.295779513° or approximately 57.3°
    Const DegPerRad As Double = 57.3
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swFeat As SldWorks.Feature
    Dim swFeatMgr As SldWorks.FeatureManager
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swChamfer As SldWorks.ChamferFeatureData2
    Dim vEdgeArr As Variant
    Dim vEdge As Variant
    Dim swEdge As SldWorks.Edge
    Dim swEnt As SldWorks.Entity
    Dim swSelData As SldWorks.SelectData
    Dim i As Long
    Dim bRet As Boolean
    Dim errors As Long
    Dim warnings As Long
    Dim fileName As String

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block20.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension

    'Create and select chamfer feature
    bRet = swModelDocExt.SelectByID("", "EDGE", -6.21980171204655E-02, 3.94066118978458E-02, -5.46079442074188E-04, True, 0, Nothing, 0)
    bRet = swModelDocExt.SelectByID("", "EDGE", -3.81606508724985E-02, 3.96345009388028E-02, 4.93384681956286E-02, True, 0, Nothing, 1)
    Set swFeatMgr = swModel.FeatureManager
    Set swFeat = swFeatMgr.InsertFeatureChamfer(4, 1, 0.00254, 0.78539816339745, 0, 0, 0, 0)
    bRet = swModelDocExt.SelectByID("Chamfer1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelMgr = swModel.SelectionManager
    Set swSelData = swSelMgr.CreateSelectData
    Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
    Set swChamfer = swFeat.GetDefinition

    'Get chamfer information
    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " " & swFeat.Name
    Debug.Print " EdgeChamferAngle = " & swChamfer.EdgeChamferAngle * DegPerRad & " degrees"
    Debug.Print " EqualDistance = " & swChamfer.EqualDistance
    Debug.Print " EdgeChamferDistance(0) = " & swChamfer.GetEdgeChamferDistance(0) * 1000# & " mm"
    Debug.Print " EdgeChamferDistance(1) = " & swChamfer.GetEdgeChamferDistance(1) * 1000# & " mm"
    Debug.Print " VertexChamferDistance(0) = " & swChamfer.GetVertexChamferDistance(0) * 1000# & " mm"
    Debug.Print " VertexChamferDistance(1) = " & swChamfer.GetVertexChamferDistance(1) * 1000# & " mm"
    Debug.Print " VertexChamferDistance(2) = " & swChamfer.GetVertexChamferDistance(2) * 1000# & " mm"
    Debug.Print " KeepFeatures = " & swChamfer.KeepFeatures
    Debug.Print " GetFaceCount = " & swChamfer.GetFaceCount
    Debug.Print " GetEdgeCount = " & swChamfer.GetEdgeCount
    Debug.Print " Type = " & swChamfer.Type
    ' ChamferFeatureData2::Type
    ' 1 = Angle-Distance
    ' 2 = Distance-Distance
    ' 3 = Vertex

    'Roll back to get access to geometric entities
    bRet = swChamfer.AccessSelections(swModel, Nothing): Debug.Assert bRet
    vEdgeArr = swChamfer.Edges

    If Not IsEmpty(vEdgeArr) Then
        swModel.ClearSelection2 True
        i = 0
        bRet = False
        For Each vEdge In vEdgeArr
            Set swEdge = vEdge
            Set swEnt = swEdge
            bRet = swEnt.Select4(True, swSelData): Debug.Assert bRet
            Debug.Print " EdgeFlip(" & i & ") = " & swChamfer.GetIsFlipped(swEdge)
            i = i + 1
        Next
    End If

    'Roll forward
    swChamfer.ReleaseSelectionAccess

End Sub

```

Get Edge Chamfer Distances Example (VB.NET)

This example shows how to get the distances of the edges and vertices for the chamfer feature.

```

-----
Preconditions:
1. Verify that the part document to open exists.
2. Open the Immediate window.

Postconditions:
1. Opens the specified part document.
2. Creates a chamfer feature.
3. Examine the Immediate window for the chamfer data.

NOTE: Because this part document is used elsewhere, do not
save any changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        ' 1 radian = 180°/p = 57.295779513° or approximately 57.3°
        Const DegPerRad As Double = 57.3
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeat As Feature
        Dim swFeatMgr As FeatureManager
        Dim swSelMgr As SelectionMgr
        Dim swChamfer As ChamferFeatureData2
        Dim vEdgeArr As Object
        Dim vEdge As Object
        Dim swEdge As Edge
        Dim swEnt As Entity
        Dim swSelData As SelectData
        Dim i As Integer
        Dim bRet As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block20.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension

        'Create and select chamfer feature
        bRet = swModelDocExt.SelectByID2("", "EDGE", -0.0621980171204655, 0.0394066118978458, -0.000546079442074188, True, 0, Nothing, 0)
        bRet = swModelDocExt.SelectByID2("", "EDGE", -0.0381606508724985, 0.0396345009388028, 0.0493384681956286, True, 0, Nothing, 1)
        swFeatMgr = swModel.FeatureManager
        swFeat = swFeatMgr.InsertFeatureChamfer(4, 1, 0.00254, 0.78539816339745, 0, 0, 0, 0)
        bRet = swModelDocExt.SelectByID2("Chamfer1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swSelMgr = swModel.SelectionManager
        swSelData = swSelMgr.CreateSelectData
        swFeat = swSelMgr.GetSelectedObject6(1, -1)
        swChamfer = swFeat.GetDefinition

        'Get chamfer information
        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print(" " & swFeat.Name)
        Debug.Print(" EdgeChamferAngle = " & swChamfer.EdgeChamferAngle * DegPerRad & " degrees")
        Debug.Print(" EqualDistance = " & swChamfer.EqualDistance)
        Debug.Print(" EdgeChamferDistance(0) = " & swChamfer.GetEdgeChamferDistance(0) * 1000.0# & " mm")
        Debug.Print(" EdgeChamferDistance(1) = " & swChamfer.GetEdgeChamferDistance(1) * 1000.0# & " mm")
        Debug.Print(" VertexChamferDistance(0) = " & swChamfer.GetVertexChamferDistance(0) * 1000.0# & " mm")
        Debug.Print(" VertexChamferDistance(1) = " & swChamfer.GetVertexChamferDistance(1) * 1000.0# & " mm")
        Debug.Print(" VertexChamferDistance(2) = " & swChamfer.GetVertexChamferDistance(2) * 1000.0# & " mm")
        Debug.Print(" KeepFeatures = " & swChamfer.KeepFeatures)
        Debug.Print(" GetFaceCount = " & swChamfer.GetFaceCount)
        Debug.Print(" GetEdgeCount = " & swChamfer.GetEdgeCount)
        Debug.Print(" Type = " & swChamfer.Type)
        ' ChamferFeatureData2::Type
        ' 1 = Angle-Distance
        ' 2 = Distance-Distance
        ' 3 = Vertex
        'Roll back to get access to geometric entities
        bRet = swChamfer.AccessSelections(swModel, Nothing) : Debug.Assert(bRet)
        vEdgeArr = swChamfer.Edges

        If Not IsNothing(vEdgeArr) Then
            swModel.ClearSelection2(True)
            i = 0
            bRet = False
            For Each vEdge In vEdgeArr
                swEdge = vEdge
                swEnt = swEdge
                bRet = swEnt.Select4(True, swSelData) : Debug.Assert(bRet)
                Debug.Print(" EdgeFlip(" & i & ") = " & swChamfer.GetIsFlipped(swEdge))
                i = i + 1
            Next
        End If

        'Roll forward
        swChamfer.ReleaseSelectionAccess()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

End Class

Get Edge Curve Parameterization Example (C#)

This example shows how to get curve parameterization data for a selected edge.

```
//-----
// Preconditions:
// 1. Open a part or assembly.
// 2. If you opened an assembly, ensure that it is
//    fully resolved.
// 3. Select an edge.
// 4. Open an Immediate Window.
//
// Postconditions: Inspect the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace GetCurveParams_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 Part = default(ModelDoc2);

            Part = (ModelDoc2)swApp.ActiveDoc;

            SelectionMgr swSelectMgr = default(SelectionMgr);
            swSelectMgr = (SelectionMgr)Part.SelectionManager;

            Edge swEdge = default(Edge);
            swEdge = (Edge)swSelectMgr.GetSelectedObject5(1);

            Curve swCurve = default(Curve);
            CurveParamData swCurveParaData = default(CurveParamData);
            swCurve = (Curve)swEdge.GetCurve();
            swCurveParaData = (CurveParamData)swEdge.GetCurveParams3();

            Debug.Print("The curve tag is: " + swCurveParaData.CurveTag);
            Debug.Print("The curve type as defined in swCurveType_e is: " + swCurveParaData.CurveType);

            double[] vEndPoint = null;
            vEndPoint = (double[])swCurveParaData.EndPoint;
            double[] EndPoint = new double[3];

            int i = 0;

            for (i = 0; i <= vEndPoint.GetUpperBound(0); i++)
            {
                EndPoint[i] = vEndPoint[i];
            }

            Debug.Print("The end point x,y,z coordinates are: " + EndPoint[0] + "," + EndPoint[1] + "," + EndPoint[2]);

            double[] vStartPoint = null;
            double[] StartPoint = new double[3];
            vStartPoint = (double[])swCurveParaData.StartPoint;
            for (i = 0; i <= vStartPoint.GetUpperBound(0); i++)
            {
                StartPoint[i] = vStartPoint[i];
            }

            Debug.Print("The start point x,y,z coordinates are: " + StartPoint[0] + "," + StartPoint[1] + "," + StartPoint[2]);

            Debug.Print("The curve and edge are in the same direction: " + swCurveParaData.Sense);
            Debug.Print("The maximum U parameter value is: " + swCurveParaData.UMaxValue);

            Debug.Print("The minimum U parameter value is: " + swCurveParaData.UMinValue);
        }
    }

    public SldWorks swApp;
}
```

Get Edge Curve Parameterization Example (VBA)

This example shows how to get curve parameterization data for a selected edge.

```
'-----
' Preconditions:
' 1. Open a part or assembly.
' 2. If you opened an assembly, ensure that it is
'    fully resolved.
' 3. Select an edge.
' 4. Open the Immediate window.
' Postconditions: Inspect the Immediate window.
'-----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks
    Dim Part As SldWorks.ModelDoc2
```

```
    Set swApp = CreateObject("SldWorks.Application")
    Set Part = swApp.ActiveDoc
```

```
    Dim swSelectMgr As SldWorks.SelectionMgr
    Set swSelectMgr = Part.SelectionManager
```

```
    Dim swEdge As SldWorks.Edge
    Set swEdge = swSelectMgr.GetSelectedObject5(1)
```

```
    Dim swCurve As SldWorks.Curve
    Dim swCurveParaData As SldWorks.CurveParamData
    Set swCurve = swEdge.GetCurve
    Set swCurveParaData = swEdge.GetCurveParams3
```

```
    Debug.Print "The curve tag is: " & swCurveParaData.CurveTag
    Debug.Print "The curve type as defined in swCurveType_e is: " & swCurveParaData.CurveType
```

```
    Dim vEndPoint As Variant
    vEndPoint = swCurveParaData.EndPoint
    Dim EndPoint(2) As Double
```

```
    Dim i As Long
```

```
    For i = 0 To UBound(vEndPoint)
        EndPoint(i) = vEndPoint(i)
    Next i
```

```
    Debug.Print "The end point x,y,z coordinates are: " & EndPoint(0) & "," & EndPoint(1) & "," & EndPoint(2)
```

```
    Dim vStartPoint As Variant
    Dim StartPoint(2) As Double
    vStartPoint = swCurveParaData.StartPoint
    For i = 0 To UBound(vStartPoint)
        StartPoint(i) = vStartPoint(i)
    Next i
```

```
    Debug.Print "The start point x,y,z coordinates are: " & StartPoint(0) & "," & StartPoint(1) & "," & StartPoint(2)
```

```
    Debug.Print "The curve and edge are in the same direction: " & swCurveParaData.Sense
    Debug.Print "The maximum U parameter value is: " & swCurveParaData.UMaxValue
    Debug.Print "The minimum U parameter value is: " & swCurveParaData.UMinValue
```

End Sub

Get Edge Curve Parameterization Example (VB.NET)

This example shows how to get curve parameterization data for a selected edge.

```
'-----
' Preconditions:
' 1. Open a part or assembly.
' 2. If you opened an assembly, ensure that it is
'    fully resolved.
' 3. Select an edge.
' 4. Open the Immediate window.
'-----
' Postconditions: Inspect the Immediate window.
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```
        Dim swApp As SldWorks
```

```
        Dim Part As ModelDoc2
```

```
        swApp = CreateObject("SldWorks.Application")
        Part = swApp.ActiveDoc
```

```
        Dim swSelectMgr As SelectionMgr
        swSelectMgr = Part.SelectionManager
```

```
        Dim swEdge As Edge
        swEdge = swSelectMgr.GetSelectedObject5(1)
```

```
        Dim swCurve As Curve
        Dim swCurveParaData As CurveParamData
        swCurve = swEdge.GetCurve
        swCurveParaData = swEdge.GetCurveParams3
```

```
        Debug.Print("The curve tag is: " & swCurveParaData.CurveTag)
        Debug.Print("The curve type as defined in swCurveType_e is: " & swCurveParaData.CurveType)
```

```
        Dim vEndPoint As Object
        vEndPoint = swCurveParaData.EndPoint
        Dim EndPoint(2) As Double
```

```
        Dim i As Long
```

```
        For i = 0 To UBound(vEndPoint)
            EndPoint(i) = vEndPoint(i)
        Next i
```

```
        Debug.Print("The end point x,y,z coordinates are: " & EndPoint(0) & "," & EndPoint(1) & "," & EndPoint(2))
```

```
        Dim vStartPoint As Object
        Dim StartPoint(2) As Double
        vStartPoint = swCurveParaData.StartPoint
        For i = 0 To UBound(vStartPoint)
            StartPoint(i) = vStartPoint(i)
        Next i
```

```
        Debug.Print("The start point x,y,z coordinates are: " & StartPoint(0) & "," & StartPoint(1) & "," & StartPoint(2))
```

```
        Debug.Print("The curve and edge are in the same direction: " & swCurveParaData.Sense)
        Debug.Print("The maximum U parameter value is: " & swCurveParaData.UMaxValue)
        Debug.Print("The minimum U parameter value is: " & swCurveParaData.UMinValue)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Get Edge Data By Name Example (C++ COM)

This example shows how to get an edge by its name and then get the underlying curve parameters and curve length. Import swconst.tlb.

IEdge::IGetCurveParams2 returns an array, so this code must be used in an in-process DLL. Otherwise, use IEdge::GetCurveParams2 that returns a VARIANT.

```
void GetEdgeData(ISldWorks* m_pSldWorks)
{
    HRESULT res = NOERROR;
    LPMODELDOC2 p_ModelDoc = NULL;
    LPPARTDOC p_PartDoc = NULL;
    LPENTITY p_Entity = NULL;
    long doctype;

    // Retrieve the model document pointer
    res = m_pSldWorks->get_IActiveDoc2( &p_ModelDoc );
    if( res != S_OK || p_ModelDoc == NULL ) return;

    res = p_ModelDoc->GetType(&doctype);
    if (doctype != swDocPART )return;

    res = p_ModelDoc->QueryInterface(IID_IPartDoc, (LPVOID *)&p_PartDoc);

    res = p_ModelDoc->GetType(&doctype);
    if (doctype != swDocPART )return;

    // Get an entity by its name "Fred'sEdge"
    res = p_PartDoc->IGetEntityByName(_T("Fred'sEdge"), swSelEDGES, &p_Entity);

    if (res == S_OK && p_Entity != NULL) // If entity was found successfully
    {
        LPEDGE p_Edge = NULL; // See if Entity is an Edge
        res = p_Entity->QueryInterface( IID_IEdge, (LPVOID*)&p_Edge);

        if (res == S_OK && p_Edge != NULL) // If Entity is an Edge
        {
            LPCURVE p_Curve;
            // Get the edge's underlying curve
            res = p_Edge->IGetCurve(&p_Curve);

            // Since we do not keep the curve parameters with every edge, this call is required or else the underlying curve parameters will be unknown.
            double edgeData[10]={NULL};
            // Get Edge data
            res = p_Edge->IGetCurveParams2(&edgeData[0]);

            // Setup for ints packed into doubles
            union PackedInts
            {
                double value;
                int intData[2];
            } packedIntData;

            // Get the integers from the double
            packedIntData.value = edgeData[8];
            int Notused = packedIntData.intData[0];
            int curveType = packedIntData.intData[1];
            // Get the integers from the double
            packedIntData.value = edgeData[9];    int Notused2 = packedIntData.intData[0];
            int curveTag = packedIntData.intData[1];

            CString message = _T("Attribute is on Edge with:\n");

            message.Format(_T("StartPt = %lf %lf %lf %lf\n EndPt = %lf %lf %lf %lf\n Params = %lf %lf %lf %lf\n CurveType = %i\n CurveTag = %i"),
                edgeData[0], edgeData[1], edgeData[2],
                edgeData[3], edgeData[4], edgeData[5],
                edgeData[6], edgeData[7],
                curveType, curveTag);
```

```
AfxMessageBox( message );

// Get the length of the underlying curve
double curveLength;
res = p_Curve->GetLength(edgeData[6], edgeData[7],
&curveLength);
message = _T("The Edge length is:\n");
message.Format(_T("%lf"), curveLength);
AfxMessageBox( message );

p_Curve->Release();

// End if we got the Edge object
}

p_Edge->Release();
p_Entity->Release();

// End if we got the Entity by its name
}

p_PartDoc->Release();
p_ModelDoc->Release();

// End of function
}
```

Get Edge Points Example (VBA)

This example shows how to get edge points.

NOTE: An edge point is a midpoint on an edge or an endpoint or midpoint on a reference curve.

```
'-----  
'  
' Preconditions: Part document is open and  
'           at least one edge point is selected.  
'  
' Postconditions: None  
'  
'-----  
  
Dim swApp As Object  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim edgep As SldWorks.EdgePoint  
    Dim swSelObj As Object  
    Dim SelType As Long  
    Dim SelCount As Long  
  
    Set swApp = GetObject(, "SldWorks.Application")  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    SelCount = swSelMgr.GetSelectedObjectCount  
  
    For i = 1 To SelCount  
        SelType = swSelMgr.GetSelectedObjectType(i)  
        Debug.Print SelType  
        If SwConst.swSelectType_e.swSelPOINTREFS = SelType Then  
            Set swSelObj = swSelMgr.GetSelectedObject5(i)  
            Set edgep = swSelObj  
        End If  
    Next i  
  
End Sub
```

Get Edges for Vertex Example (VBA)

This example shows how to get a list of the edges that meet at the selected vertex.

```
'-----  
' Preconditions: Open a part and select a vertex.  
'  
' Postconditions:  
' 1. Selects all edges that meet at the selected vertex.  
' 2. Examine the graphics area.  
'-----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swVert As SldWorks.Vertex  
    Dim vEdgeArr As Variant  
    Dim vEdge As Variant  
    Dim swEdge As SldWorks.Edge  
    Dim swEnt As SldWorks.Entity  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swVert = swSelMgr.GetSelectedObject6(1, -1)  
  
    swModel.ClearSelection2 True  
  
    vEdgeArr = swVert.GetEdges  
    For Each vEdge In vEdgeArr  
        Set swEdge = vEdge  
        Set swEnt = swEdge  
        bRet = swEnt.Select4(True, Nothing)  
    Next vEdge
```

End Sub

Detect In-Context Edit Example (C++ COM)

This example shows how to trap an `swAssemblyBeginInContextEditNotify` event and examine the document being edited in an assembly.

1. Create a new C++ COM add-in with the Visual Studio.NET COM AppWizard.
2. Select **Assembly Events** on the **SwOptions** page.
3. After the wizard finishes generating code, add this code to the specified files.

a. `swDocument.h`:

```
SINK_ENTRY_EX(ID_ASSEMBLY_EVENTS, __uuidof(DAssemblyDocEvents), swAssemblyBeginInContextEditNotify, OnAssemblyBeginInContextEditNotify)

STDMETHOD(OnAssemblyBeginInContextEditNotify)(LPDISPATCH newDoc, long docType); //
next to the other SINK_ENTRY_EX and STDMETHOD Declarations
```

b. `swDocument.cpp`:

```
STDMETHODIMP CSwDocument::OnAssemblyBeginInContextEditNotify(LPDISPATCH newDoc, long docType)
{
    ATLTRACE("tCCSwDocument::OnAssemblyBeginInContextEditNotify called\n");
    CComPtr<IModelDoc2> swModel;
    ISwApp->get_IActiveDoc2(&swModel);
    CComPtr<IModelDoc2> swNewModel;
    newDoc->QueryInterface(__uuidof(IModelDoc2), reinterpret_cast<void*>(&swNewModel));

    CComBSTR ContextName;
    CComBSTR MainName;
    swNewModel->GetTitle(&ContextName);
    swModel->GetTitle(&MainName);
    CComBSTR message;
    message.Append(OLESTR("Main Assembly: "));
    message.Append(MainName);
    message.Append(OLESTR(" Component: "));
    message.Append(ContextName);
    long res;
    ISwApp->SendMsgToUser2(message, 0, 0, &res);

    return S_OK;
}
```

4. After loading the add-in, open an assembly, right-click a component, and select **Edit Part** or **Edit Subassembly**. The new method `OnAssemblyBeginInContextEditNotify` that you implemented should be called.

Get Editing Status of Features Example (C#)

This example shows how to get the editing status of one or more features.

```
//-----
// Preconditions
// 1. Open public_documents\samples\tutorial\introsw\pressure_plate.sldprt.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. At System.Diagnostics.Debugger.Break(), examine the the Immediate
// window, graphics area, and FeatureManager design tree.
// 2. Press F5.
// 3. Examine the Immediate window again.
//
// NOTE: Because this document is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
using System.Runtime.InteropServices;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            FeatureManager swFeatMgr = default(FeatureManager);
            SelectionMgr swSelMgr = default(SelectionMgr);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Object[] varFeat;
            long editStatus = 0;
            bool retVal = false;
            long i = 0;
            string featName = null;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swFeatMgr = swModel.FeatureManager;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swModelDocExt = swModel.Extension;

            // Traverse through the FeatureManager design tree
            // to get the editing status of all features
            // Change the editing status of a sketch and feature
            // during feature traversal
            varFeat = (Object[])swFeatMgr.GetFeatures(true);
            editStatus = (long)swFeatureEditStatus_e.swFeature_NonEditable;
            for (i = varFeat.GetLowerBound(0); i <= varFeat.GetUpperBound(0); i++)
            {
                Feature swFeat = default(Feature);
                swFeat = (Feature)varFeat[i];
                featName = swFeat.Name;
                switch ((featName))
                {
                    case "Sketch2":
                        // Select and edit a sketch
                        retVal = swModelDocExt.SelectByID("Sketch2", "SKETCH", 0, 0, 0, false, 0, null, 0);
                        swModel.EditSketch();

                        System.Diagnostics.Debugger.Break();
                        // Examine the Immediate window, graphics area, and FeatureManager design tree
                        // All of the features beneath Extrude1 cannot be edited because
                        // Extrude2's Sketch2 is in edit mode
                        // Press F5

                        break;

                    case "Extrude3":
                        // Close the open sketch
                        swModel.InsertSketch2(true);
                        break;

                    case "Cut-Extrude2":
                        // Select and edit a feature
                        retVal = swModelDocExt.SelectByID("Cut-Extrude2", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
                        swModel.FeatEdit();
                        break;

                }

                // Get the editing status of the current feature
                editStatus = swFeat.GetEditStatus();
                switch ((editStatus))
                {
                    case 0:
                        Debug.Print(swFeat.Name + " can be edited.");
                        break;
                    case 1:
                        Debug.Print(swFeat.Name + " cannot currently be edited.");
                        break;
                    case 2:
                        Debug.Print(swFeat.Name + " is already being edited.");
                        break;
                }
                swFeat = null;
            }

            // End feature editing
            swModel.InsertSketch2(true);
        }
        public SldWorks swApp;
    }
}
```

Get Editing Status of Features Example (VBA)

This example shows how to get the editing status of one or more features.

```
'-----
' Preconditions
' 1. Open public_documents\samples\tutorial\introsw\pressure_plate.sldprt.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. At Stop, examine the the Immediate window, graphics area, and
'   FeatureManager design tree.
' 2. Press F5.
' 3. Examine the Immediate window again.
'
' NOTE: Because this document is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swFeatMgr As SldWorks.FeatureManager
Dim swSelMgr As SldWorks.SelectionMgr
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim varFeat As Variant
Dim editStatus As Long
Dim retVal As Boolean
Dim i As Long
Dim featName As String

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swFeatMgr = swModel.FeatureManager
    Set swSelMgr = swModel.SelectionManager
    Set swModelDocExt = swModel.Extension

    ' Traverse through the FeatureManager design tree
    ' to get the editing status of all features
    ' Change the editing status of a sketch and feature
    ' during feature traversal
    varFeat = swFeatMgr.GetFeatures(True)
    editStatus = swFeature.NonEditable
    For i = LBound(varFeat) To UBound(varFeat)
        Dim swFeat As SldWorks.Feature
        Set swFeat = varFeat(i)
        featName = swFeat.Name
        Select Case (featName)
            Case "Sketch2"
                ' Select and edit a sketch
                retVal = swModelDocExt.SelectByID("Sketch2", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
                swModel.EditSketch

                Stop
                ' Examine the Immediate window, graphics area, and FeatureManager design tree
                ' All of the features beneath Extrude1 cannot be edited because
                ' Extrude2's Sketch2 is in edit mode
                ' Press F5

            Case "Extrude3"
                ' Close the open sketch
                swModel.InsertSketch2 True
            Case "Cut-Extrude2"
                ' Select and edit a feature
                retVal = swModelDocExt.SelectByID("Cut-Extrude2", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
                swModel.FeatEdit
        End Select

        ' Get the editing status of the current feature
        editStatus = swFeat.GetEditStatus
        Select Case (editStatus)
            Case 0
                Debug.Print (swFeat.Name & " can be edited.")
            Case 1
                Debug.Print (swFeat.Name & " cannot currently be edited.")
            Case 2
                Debug.Print (swFeat.Name & " is already being edited.")
        End Select

        Set swFeat = Nothing
    Next i

    ' End feature editing
    swModel.InsertSketch2 True
End Sub
```

Get Editing Status of Features Example (VB.NET)

This example shows how to get the editing status of one or more features.

```

-----
Preconditions
1. Open public_documents\samples\tutorial\introtosw\pressure_plate.sldprt.
2. Open the Immediate window.

Postconditions:
1. At Stop, examine the the Immediate window, graphics area, and
   FeatureManager design tree.
2. Press F5.
3. Examine the Immediate window again.

NOTE: Because this document is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swFeatMgr As FeatureManager
        Dim swSelMgr As SelectionMgr
        Dim swModelDocExt As ModelDocExtension
        Dim varFeat As Object
        Dim editStatus As Long
        Dim retVal As Boolean
        Dim i As Long
        Dim featName As String

        swModel = swApp.ActiveDoc
        swFeatMgr = swModel.FeatureManager
        swSelMgr = swModel.SelectionManager
        swModelDocExt = swModel.Extension

        ' Traverse through the FeatureManager design tree
        ' to get the editing status of all features
        ' Change the editing status of a sketch and feature
        ' during feature traversal
        varFeat = swFeatMgr.GetFeatures(True)
        editStatus = swFeatureEditStatus_e.swFeature_NonEditable
        For i = LBound(varFeat) To UBound(varFeat)
            Dim swFeat As Feature
            swFeat = varFeat(i)
            featName = swFeat.Name
            Select Case (featName)
                Case "Sketch2"
                    ' Select and edit a sketch
                    retVal = swModelDocExt.SelectById2("Sketch2", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
                    swModel.EditSketch()

                    Stop
                    ' Examine the Immediate window, graphics area, and FeatureManager design tree
                    ' All of the features beneath Extrude1 cannot be edited because
                    ' Extrude2's Sketch2 is in edit mode
                    ' Press F5

                Case "Extrude3"
                    ' Close the open sketch
                    swModel.InsertSketch2(True)

                Case "Cut-Extrude2"
                    ' Select and edit a feature
                    retVal = swModelDocExt.SelectById2("Cut-Extrude2", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
                    swModel.FeatEdit()
            End Select
            ' Get the editing status of the current feature
            editStatus = swFeat.GetEditStatus
            Select Case (editStatus)
                Case 0
                    Debug.Print(swFeat.Name & " can be edited.")
                Case 1
                    Debug.Print(swFeat.Name & " cannot currently be edited.")
                Case 2
                    Debug.Print(swFeat.Name & " is already being edited.")
            End Select
            swFeat = Nothing
        Next i
        ' End feature editing
        swModel.InsertSketch2(True)

    End Sub

    ''' <summary>
    ''' The Sldworks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get Entities Attached to Cosmetic Thread Example (VBA)

This example shows how to locate all cosmetic thread annotations in the drawing view, obtain the associated entities, and query those entities for their radius values.

```

'-----
' Preconditions:
' 1. Open a drawing of a model with a cosmetic thread.
' 2. Select a drawing view of the model with the cosmetic
'    thread.
'
' Postconditions: Displays a message box with the
' radius of the cosmetic thread.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim activeDocument As SldWorks.DrawingDoc
    Dim pAnnotation As SldWorks.Annotation
    Dim selType As Long
    Dim entityCount As Long
    Dim SelMgr As Object
    Dim entitiesArray As Variant
    Dim attachedEntity As SldWorks.Entity
    Dim selView As SldWorks.View
    Dim annotType As Long
    Dim attachedEntityTypes As Variant
    Dim i As Long
    Dim edgeParams As Variant
    Dim edgeRadius As Double
    Dim faceParams As Variant
    Dim faceRadius As Double

    Set swApp = CreateObject("SldWorks.Application")
    Set activeDocument = swApp.ActiveDoc

    If (activeDocument.GetType <> swDocDRAWING) Then
        Exit Sub
    End If

    Set SelMgr = activeDocument.SelectionManager
    selType = SelMgr.GetSelectedObjectType2(1)

    If (selType = swSelDRAWINGVIEWS) Then
        ' Get the selected drawing view
        Set selView = SelMgr.GetSelectedObject6(1, -1)
        If (selView Is Nothing) Then
            Exit Sub
        End If
        ' Get first annotation in the view
        Set pAnnotation = selView.GetFirstAnnotation3
        ' Loop through every annotation in view
        While (Not pAnnotation Is Nothing)
            ' Get the type of annotation
            annotType = pAnnotation.GetType
            ' For each CThread, get radius of attached entity
            If (annotType = swCThread) Then
                ' Get attached entities
                entitiesArray = pAnnotation.GetAttachedEntities
                attachedEntityTypes = pAnnotation.GetAttachedEntityTypes
                ' Get number of attached entities
                entityCount = UBound(entitiesArray) + 1
                ' For each entity attached to this CThread
                For i = 0 To (entityCount - 1)
                    ' Get the individual Edge or Face2 object
                    Set attachedEntity = entitiesArray(i)
                    ' If attached entity is an edge
                    If (attachedEntityTypes(i) = swSelEDGES) Then
                        edgeParams = attachedEntity.GetCurve.CircleParams
                        edgeRadius = edgeParams(6)
                        swApp.SendMsgToUser "Cosmetic Thread Edge Radius = " + Str(edgeRadius)
                    ' If attached entity is a face
                    ElseIf (attachedEntityTypes(i) = swSelFACES) Then
                        faceParams = attachedEntity.GetSurface.CylinderParams
                        faceRadius = faceParams(6)
                        swApp.SendMsgToUser "Cosmetic Thread Face Radius = " + Str(faceRadius)
                    End If
                Next i
            End If
            Set pAnnotation = pAnnotation.GetNext3
        Wend
    Else
        swApp.SendMsgToUser "Please select a drawing view containing cosmetic thread annotations"
    End If
End Sub

```

Get Entity By Name Example (VBA)

This example shows how to get a face by specifying its name and changing the name. This process is commonly used with standardized parts where names can be given to objects and the objects can be accessed later.

```
-----  
' Preconditions:  
' 1. Open a part document.  
' 2. Select a face.  
'  
' Postconditions:  
' 1. Displays a message box containing the  
'   name of the selected face.  
' 2. Click OK.  
' 3. Displays a message box containing  
'   the new name of the selected face.  
' 4. Click OK.  
-----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks  
    Dim Part As SldWorks.PartDoc  
    Dim Face As SldWorks.Face2  
    Dim Model As SldWorks.ModelDoc2  
    Dim ModelDocExt As SldWorks.ModelDocExtension  
    Dim SelMgr As SldWorks.SelectionMgr  
    Dim Entity As SldWorks.Entity  
    Dim faceName As String  
    Dim ret As Boolean  
  
    Set swApp = CreateObject("SldWorks.Application")  
  
    ' Get active document  
    Set Part = swApp.ActiveDoc  
    Set Model = Part  
  
    ' Get face name  
    Set SelMgr = Model.SelectionManager  
    Set Face = SelMgr.GetSelectedObject6(1, -1)  
    Set Entity = Face  
    faceName = Model.GetEntityName(Entity)  
  
    swApp.SendMsgToUser "Selected face name: " & faceName  
  
    ' Get face by its name  
    Set Face = Part.GetEntityByName(faceName, 2)  
  
    ' Delete current face name  
    ret = Part.DeleteEntityName(Entity)  
  
    ' Change name of the face  
    ret = Part.SetEntityName(Entity, "NewFaceName")  
  
    ' If name change is successful  
    If ret Then  
        swApp.SendMsgToUser "Selected face name changed to: " & Model.GetEntityName(Entity)  
    ' If name change failed  
    Else  
        swApp.SendMsgToUser "Error changing face name."  
    End If  
End Sub
```

Get Entity Name Example (C++ COM)

This example shows how to get the name of the IEntity object.

```
BSTR entityName;  
res = m_PartDoc->IGetEntityName(m_Entity, &entityName);  
  
CString message;  
message.Format(_T("Entity Name is: \t%s"), entityName);  
AfxMessageBox( message );  
  
SysFreeString(entityName);
```

Get Equation Values Example (C#)

This example shows how to get the values of equations.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\partequations.sldprt.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Gets each equation's value and index and whether the
//    equation is a global variable.
// 2. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            EquationMgr swEqnMgr = default(EquationMgr);
            int i = 0;
            int nCount = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swEqnMgr = (EquationMgr)swModel.GetEquationMgr();
            Debug.Print("File = " + swModel.GetPathName());
            nCount = swEqnMgr.GetCount();
            for (i = 0; i < nCount; i++)
            {
                Debug.Print("    Equation(" + i + ") = " + swEqnMgr.get_Equation(i));
                Debug.Print("    Value = " + swEqnMgr.get_Value(i));
                Debug.Print("    Index = " + swEqnMgr.Status);
                Debug.Print("    Global variable? " + swEqnMgr.get_GlobalVariable(i));
            }
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}
```

Get Equation Values Example (VBA)

This example shows how to get the values of equations.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\partequations.sldprt.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets each equation's value and index and whether the  
'    equation is a global variable.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swEqnMgr As SldWorks.EquationMgr  
    Dim i As Long  
    Dim nCount As Long  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swEqnMgr = swModel.GetEquationMgr  
  
    Debug.Print "File = " & swModel.GetPathName  
  
    nCount = swEqnMgr.GetCount  
    For i = 0 To nCount - 1  
        Debug.Print "    Equation(" & i & ") = " & swEqnMgr.Equation(i)  
        Debug.Print "        Value = " & swEqnMgr.Value(i)  
        Debug.Print "        Index = " & swEqnMgr.Status  
        Debug.Print "        Global variable? " & swEqnMgr.GlobalVariable(i)  
    Next i  
  
End Sub
```

Get Equation Values Example (VB.NET)

This example shows how to get the values of equations.

```
-----
Preconditions:
1. Open public_documents\samples\tutorial\api\partequations.sldprt.
2. Open the Immediate window.
Postconditions:
1. Gets each equation's value and index and whether the
   equation is a global variable.
2. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swEqnMgr As EquationMgr
        Dim i As Integer
        Dim nCount As Integer

        swModel = swApp.ActiveDoc
        swEqnMgr = swModel.GetEquationMgr
        Debug.Print("File = " & swModel.GetPathName)
        nCount = swEqnMgr.GetCount
        For i = 0 To nCount - 1
            Debug.Print("    Equation(" & i & ") = " & swEqnMgr.Equation(i))
            Debug.Print("        Value = " & swEqnMgr.Value(i))
            Debug.Print("        Index = " & swEqnMgr.Status)
            Debug.Print("        Global variable? " & swEqnMgr.GlobalVariable(i))
        Next i

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Get Error Codes for Features (VBA)

This example shows how to get the error codes and warnings for the features in a part document.

```
'-----  
' Preconditions:  
' 1. Open a part.  
' 2. Open the Immediate window.  
'  
' Postconditions: Examine the Immediate window.  
'  
' NOTE: Features with errors have error codes of non-zero.  
' Only examine warnings if error codes are non-zero.  
' Otherwise, values for warnings are irrelevant.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim Model As SldWorks.ModelDoc2  
Dim feat As SldWorks.Feature  
Dim featureName As String  
Dim IsWarning As Boolean  
Dim Error As Long  
  
Sub main()  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set Model = swApp.ActiveDoc  
  
    If Model Is Nothing Then  
        Exit Sub  
    End If  
  
    ' Get the first feature in part  
    Set feat = Model.FirstFeature  
    ' While we have a valid feature  
    Do While Not feat Is Nothing  
        ' Get the name of the feature  
        Debug.Print feat.Name  
        Error = feat.GetErrorCode2(IsWarning)  
        Debug.Print "Error:      " & Error  
        Debug.Print "Warning:    " & IsWarning  
        Set feat = feat.GetNextFeature() ' Get the next feature  
    Loop ' Continue until no more features exist  
  
End Sub
```

Get Excel Cell Value for Density Example (VBA)

This example shows how to get a density value from an active Excel worksheet and use that density for the active part.

```
'-----  
Sub main()  
  
  ' Define variable used to hold the ISldWorks object  
  Dim swApp As Object  
  
  ' Define variable used to hold the IModelDoc object  
  Dim Part As Object  
  
  ' Constant enumerators  
  Const swDocPART = 1  
  Const swMaterialPropertyDensity = 7  
  
  Set swApp = CreateObject ("SldWorks.Application")  
  
  ' Get handle to the active SOLIDWORKS Design part  
  Set Part = swApp.ActiveDoc  
  
  ' If not a part, then exit  
  If (Part.GetType <> swDocPART) Then  
    Exit Sub  
  End If  
  
  ' Attaches to the active Excel object  
  Set xl = GetObject(,"Excel.Application")  
  
  ' Get handle to the active sheet in Excel  
  Set xlsh = xl.ActiveSheet  
  
  ' Gets value in cell A1  
  density = xlsh.Cells(1,1)  
  
  ' Set the density in the SOLIDWORKS part  
  Part.SetUserPreferenceDoubleValue swMaterialPropertyDensity, density  
  
End Sub
```



```

For j = 1 To tableColCount
    colIndex = tableFirstCol + j - 1
    Set cellRange = myWorksheet.Cells(rowIndex, colIndex)
    cellValue = cellRange.Value2
    cellText = cellRange.Text
    horzalign = cellRange.HorizontalAlignment
    Select Case horzalign
    Case XLHAlign.xlHAlignLeft
        halignStr = "Left"
    Case XLHAlign.xlHAlignCenter
        halignStr = "Center"
    Case XLHAlign.xlHAlignRight
        halignStr = "Right"
    Case XLHAlign.xlHAlignCenterAcrossSelection
        halignStr = "Center Across"
    Case XLHAlign.xlHAlignDistributed
        halignStr = "Distributed"
    Case Else
        halignStr = "unknown"
    End Select
    vertalign = cellRange.VerticalAlignment
    Select Case vertalign
    Case XLVAlign.xlVAlignBottom
        valignStr = "Bottom"
    Case XLVAlign.xlVAlignCenter
        valignStr = "Middle"
    Case XLVAlign.xlVAlignTop
        valignStr = "Top"
    Case XLVAlign.xlVAlignDistributed
        valignStr = "Distributed"
    Case Else
        valignStr = "unknown"
    End Select
    ismerged = cellRange.MergeCells
    Debug.Print "Cell[" & rowIndex & ", " & colIndex & "] = " & cellText & " " & cellValue & " " & halignStr & " " & valignStr & "
" & ismerged
        Next j
    Next i
End If
Set myWorksheet = Nothing
myDesignTable.Detach
End If
swApp.CloseDoc viewModelName
End If
Set myView = myView.GetNextView()
Wend
End Sub

```

Get Explode Step Component Transform Example (VBA)

This example shows how to get the transformation matrix of a component in a radial explode step and apply it to another component to overlap the components.

```

'-----
' Preconditions:
' 1. Open an assembly with an explode view with at least one radial explode
'   step with at least two components.
' 2. Activate the explode view.
' 3. Replace component_name in the code below with the name of a second
'   component to overlap the first.
' 4. Open an Immediate window.
'
' Postconditions: For each explode step:
' 1. Gets the transformation matrix of a component in the radial explode step.
' 2. Selects another component to overlap.
' 3. Applies the transformation matrix to overlap the components.
' 4. Prints to the Immediate window the transformation matrix for the component.
'-----

```

```

Dim swApp As SldWorks.SldWorks
Dim assDoc As SldWorks.AssemblyDoc
Dim c As SldWorks.Configuration
Dim Part As MSldWorks.odelDoc2
Dim es As SldWorks.ExplodeStep
Dim comp As SldWorks.Component2
Dim mathXForm As SldWorks.MathTransform
Dim compXForm As SldWorks.MathTransform
Dim obj As SldWorks.Component2
Dim selMgr As SSldWorks.electionMgr
Dim viewCount, nStep, i, j, k As Long
Dim varComp, varXForm As Variant
Dim MathUtility As SldWorks.MathUtility
Dim boolstatus As Boolean
Dim longstatus As Long, longwarnings As Long
Option Explicit
Sub main()
    Set swApp = Application.SldWorks
    Set Part = swApp.ActiveDoc
    Set assDoc = Part

    Set c = Part.GetActiveConfiguration

    nStep = c.GetNumberOfExplodeSteps

    For i = 0 To nStep - 1
        j = 1

        Set es = c.GetExplodeStep(i)
        varComp = es.GetComponents

        'Get the transformation matrix for the second component in the radial explode step
        Set comp = es.GetComponent(j)
        Debug.Print comp.Name2
        varXForm = es.GetStepComponentXForm(j)
        Set MathUtility = swApp.GetMathUtility
        Set mathXForm = MathUtility.CreateTransform(varXForm)

        'Select another component to overlap
        Set selMgr = Part.SelectionManager
        boolstatus = Part.Extension.SelectByID2("component_name", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        Set obj = selMgr.GetSelectedObject6(1, -1)

        'Applies the transformation matrix to overlap the components
        Set compXForm = obj.Transform2
        Set compXForm = compXForm.Multiply(mathXForm)
        obj.Transform2 = compXForm

        Debug.Print es.Name & " : "
        For k = 0 To 15
            Debug.Print varXForm(k)
        Next k

        Call Part.EditRebuild3

    Next i
End Sub

```

Get Explode Step Example (C#)

This example shows how to get an explode step and its properties.

```
//-----
// Preconditions:
// 1. Open an assembly document with an explode view.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Gets the first explode step.
// 2. Examine the Immediate window.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace ExplodeStepCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ConfigurationManager swConfigurationMgr = default(ConfigurationManager);
            Configuration swConfiguration = default(Configuration);
            ExplodeStep swExplodeStep = default(ExplodeStep);

            swModel = (ModelDoc2)swApp.ActiveDoc;

            //Get explode step
            swConfigurationMgr = (ConfigurationManager)swModel.ConfigurationManager;
            swConfiguration = (Configuration)swConfigurationMgr.ActiveConfiguration;
            swExplodeStep = (ExplodeStep)swConfiguration.GetExplodeStep(0);
            Debug.Print("Name of explode step: " + swExplodeStep.Name);
            Debug.Print("Number of components that move in this explode step: " + swExplodeStep.GetNumOfComponents());
            Debug.Print("Is the sub-assembly rigid? " + swExplodeStep.IsSubAssemblyRigid());
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get Explode Step Example (VBA)

This example shows how to get an explode step and its properties.

```
'-----  
' Preconditions:  
' 1. Open an assembly document with an explode view.  
' 2. Open an Immediate window.  
'  
' Postconditions:  
' 1. Gets the first explode step.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swConfigurationMgr As SldWorks.ConfigurationManager  
Dim swConfiguration As SldWorks.Configuration  
Dim swExplodeStep As SldWorks.ExplodeStep  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    'Get explode step  
    Set swConfigurationMgr = swModel.ConfigurationManager  
    Set swConfiguration = swConfigurationMgr.ActiveConfiguration  
    Set swExplodeStep = swConfiguration.GetExplodeStep(0)  
    Debug.Print "Name of explode step: " & swExplodeStep.Name  
    Debug.Print "Number of components that move in this explode step: " & swExplodeStep.GetNumOfComponents  
    Debug.Print "Is the sub-assembly rigid? " & swExplodeStep.IsSubAssemblyRigid  
  
End Sub
```

Get Explode Step Example (VB.NET)

This example shows how to get an explode step and its properties.

```
-----
' Preconditions:
' 1. Open an assembly document with an explode view.
' 2. Open an Immediate window.
' Postconditions:
' 1. Gets the first explode step.
' 2. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swConfigurationMgr As ConfigurationManager
        Dim swConfiguration As Configuration
        Dim swExplodeStep As ExplodeStep

        swModel = swApp.ActiveDoc

        'Get explode step
        swConfigurationMgr = swModel.ConfigurationManager
        swConfiguration = swConfigurationMgr.ActiveConfiguration
        swExplodeStep = swConfiguration.GetExplodeStep(0)
        Debug.Print("Name of explode step: " & swExplodeStep.Name)
        Debug.Print("Number of components that move in this explode step: " & swExplodeStep.GetNumOfComponents)
        Debug.Print("Is the sub-assembly rigid? " & swExplodeStep.IsSubAssemblyRigid)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Get Exploded Views for Configuration Example (C#)

This example shows how to get:

- number of exploded views for a configuration.
- name of each exploded view for a configuration.
- name of the configuration for each exploded view.
- name of the exploded view shown in the model.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\pdmworks\speaker.sldasm.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Gets the name of the active configuration.
// 2. Creates five exploded views for the active configuration.
// 3. Gets the number of exploded views for the active configuration.
// 4. Gets the name of:
//    * each exploded view for the active configuration
//    * configuration for each exploded view
//    and shows each exploded view.
// 5. Gets the name of the exploded view shown in the model.
// 6. Examine the Immediate window.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            AssemblyDoc swAssembly = default(AssemblyDoc);
            ConfigurationManager swConfigMgr = default(ConfigurationManager);
            Configuration swConfig = default(Configuration);
            string activeConfigName = null;
            string[] viewNames = null;
            string viewName = null;
            int i = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssembly = (AssemblyDoc)swModel;

            //Get active configuration name
            swConfigMgr = (ConfigurationManager)swModel.ConfigurationManager;
            swConfig = (Configuration)swConfigMgr.ActiveConfiguration;
            activeConfigName = swConfig.Name;

            Debug.Print("Active configuration name: " + activeConfigName);

            //Create five exploded views in the active configuration
            for (i = 0; i <= 4; i++)
            {
                swAssembly.CreateExplodedView();
            }

            //Get the number of exploded views in the active configuration name
            Debug.Print(" Number of exploded views created: " + swAssembly.GetExplodedViewCount2(activeConfigName));

            //Get the name of each exploded view in the active configuration,
            //get the name of the configuration for each exploded view, and
            //show each exploded view
            viewNames = (string[])swAssembly.GetExplodedViewNames2(activeConfigName);

            for (i = 0; i < viewNames.Length; i++)
            {
                viewName = viewNames[i];
                Debug.Print(" Exploded view name: " + viewName);
                Debug.Print(" Name of configuration for exploded view: " + swAssembly.GetExplodedViewConfigurationName(viewName));
                swAssembly.ShowExploded2(true, viewName);
            }

            //Get the name of exploded view shown in model
            viewName = "";
            swModelDocExt = swModel.Extension;
            swModelDocExt.IsExploded(out viewName);
            Debug.Print("Name of exploded view shown in model: " + viewName);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get Exploded Views for Configuration Example (VBA)

This example shows how to get:

- number of exploded views for a configuration.
- name of each exploded view for a configuration.
- name of the configuration for each exploded view.
- name of the exploded view shown in the model.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\pdmworks\speaker.sldasm.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets the name of the active configuration.
' 2. Creates five exploded views for the active configuration.
' 3. Gets the number of exploded views for the active configuration.
' 4. Gets the name of:
'    * each exploded view for the active configuration
'    * configuration for each exploded view
'    and shows each exploded view.
' 5. Gets the name of the exploded view shown in the model.
' 6. Examine the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Option Explicit
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swAssembly As SldWorks.AssemblyDoc
Dim swConfigMgr As SldWorks.ConfigurationManager
Dim swConfig As SldWorks.Configuration
Dim activeConfigName As String
Dim viewNames As Variant
Dim viewName As String
Dim i As Long

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swAssembly = swModel

    'Get active configuration name
    Set swConfigMgr = swModel.ConfigurationManager
    Set swConfig = swConfigMgr.ActiveConfiguration
    activeConfigName = swConfig.Name

    Debug.Print "Active configuration name: " & activeConfigName

    'Create five exploded views in the active configuration
    For i = 0 To 4
        swAssembly.CreateExplodedView
    Next i

    'Get the number of exploded views in the active configuration name
    Debug.Print "    Number of exploded views created: " & swAssembly.GetExplodedViewCount2(activeConfigName)

    'Get the name of each exploded view in the active configuration,
    'get the name of the configuration for each exploded view, and
    'show each exploded view
    viewNames = swAssembly.GetExplodedViewNames2(activeConfigName)

    For i = 0 To UBound(viewNames)
        viewName = viewNames(i)
        Debug.Print "    Exploded view name: " & viewName
        Debug.Print "    Name of configuration for exploded view: " & swAssembly.GetExplodedViewConfigurationName(viewName)
        swAssembly.ShowExploded2 True, viewName
    Next i

    'Get the name of exploded view shown in model
    viewName = ""
    Set swModelDocExt = swModel.Extension
    swModelDocExt.IsExploded viewName
    Debug.Print "Name of exploded view shown in model: " & viewName

End Sub

```

Get Exploded Views for Configuration Example (VB.NET)

This example shows how to get:

- number of exploded views for a configuration.
- name of each exploded view for a configuration.
- name of the configuration for each exploded view.
- name of the exploded view shown in the model.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\pdmworks\speaker.sldasm.
' 2. Open the Immediate window.
'-----
' Postconditions:
' 1. Gets the name of the active configuration.
' 2. Creates five exploded views for the active configuration.
' 3. Gets the number of exploded views for the active configuration.
' 4. Gets the name of:
'    * each exploded view for the active configuration
'    * configuration for each exploded view
'    and shows each exploded view.
' 5. Gets the name of the exploded view shown in the model.
' 6. Examine the Immediate window.
'-----
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swAssembly As AssemblyDoc
        Dim swConfigMgr As ConfigurationManager
        Dim swConfig As Configuration
        Dim activeConfigName As String
        Dim viewNames() As String
        Dim viewName As String
        Dim i As Integer

        swModel = swApp.ActiveDoc
        swAssembly = swModel

        'Get active configuration name
        swConfigMgr = swModel.ConfigurationManager
        swConfig = swConfigMgr.ActiveConfiguration
        activeConfigName = swConfig.Name

        Debug.Print("Active configuration name: " & activeConfigName)

        'Create five exploded views in the active configuration
        For i = 0 To 4
            swAssembly.CreateExplodedView()
        Next i

        'Get the number of exploded views in the active configuration name
        Debug.Print("    Number of exploded views created: " & swAssembly.GetExplodedViewCount2(activeConfigName))

        'Get the name of each exploded view in the active configuration,
        'get the name of the configuration for each exploded view, and
        'show each exploded view
        viewNames = swAssembly.GetExplodedViewNames2(activeConfigName)

        For i = 0 To UBound(viewNames)
            viewName = viewNames(i)
            Debug.Print("    Exploded view name: " & viewName)
            Debug.Print("    Name of configuration for exploded view: " & swAssembly.GetExplodedViewConfigurationName(viewName))
            swAssembly.ShowExploded2(True, viewName)
        Next i

        'Get the name of exploded view shown in model
        viewName = ""
        swModelDocExt = swModel.Extension
        swModelDocExt.IsExploded(viewName)
        Debug.Print("Name of exploded view shown in model: " & viewName)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get Extension Line Direction Example (VBA)

This example shows how to get the direction of the selected dimension's extension line.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\advdrawings\foodprocessor.slddrw.
' 2. Select a non-radial dimension.
'
' Postconditions:
' 1. Gets the extension line direction for the selected non-radial
'    dimension.
' 2. Examine the Immediate window.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swDim As SldWorks.Dimension
    Dim swDispDim As SldWorks.DisplayDimension
    Dim swExtLineDirn As SldWorks.MathVector
    Dim i As Long
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swDispDim = swSelMgr.GetSelectedObject6(1, -1)
    Set swDim = swDispDim.GetDimension

    Set swExtLineDirn = swDim.ExtensionLineDirection

    ' IDimension::ExtensionLineDirection only returns
    ' non-0 vector for feature dimensions
    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " Name = " & swDim.FullName
    Debug.Print " Extension line direction = (" & swExtLineDirn.ArrayData(0) & ", " & swExtLineDirn.ArrayData(1) & ", " & swExtLineDirn.ArrayData(2) &
    ")"

End Sub
```



```

Status = (int[])vStatus;
RefEntity = (object[])vRefEntity;
FeatComp = (object[])vFeatComp;

Debug.Print("");
for (i = 0; i <= nRefCount - 1; i++)
{
    Debug.Print("    Model path + name      = " + ModelPathName[i]);
    Debug.Print("    Component path + name    = " + ComponentPathName[i]);
    Debug.Print("    Feature                      = " + Feature[i]);
    Debug.Print("    Data type                   = " + DataType[i]);
    Debug.Print("    Status                      = " + System.Convert.ToString(Status[i]));
    Debug.Print("    Reference entity            = " + RefEntity[i]);
    Debug.Print("    Feature component           = " + FeatComp[i]);

    Debug.Print(" ");
}
Debug.Print("    Config option          = " + nConfigOpt);
Debug.Print("    Config name           = " + sConfigName);
}

break;
// Selected feature in a part or assembly document
case (int)swSelectType_e.swSelBODYFEATURES:
case (int)swSelectType_e.swSelSKETCHES:
    swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
    nRefCount = swFeat.ListExternalFileReferencesCount();
    swFeat.ListExternalFileReferences2(out vModelPathName, out vComponentPathName, out vFeature, out vDataType, out
vStatus, out vRefEntity, out vFeatComp, out nConfigOpt, out sConfigName);

    Debug.Print("Model name = " + swModel.GetPathName());
    Debug.Print("    Reference count      = " + System.Convert.ToString(nRefCount));

    if (nRefCount >= 1)
    {
        object[] ModelPathName = new object[nRefCount - 1];
        object[] ComponentPathName = new object[nRefCount - 1];
        object[] Feature = new object[nRefCount - 1];
        object[] DataType = new object[nRefCount - 1];
        int[] Status = new int[nRefCount - 1];
        object[] RefEntity = new object[nRefCount - 1];
        object[] FeatComp = new object[nRefCount - 1];

        ModelPathName = (object[])vModelPathName;
        ComponentPathName = (object[])vComponentPathName;
        Feature = (object[])vFeature;
        DataType = (object[])vDataType;
        Status = (int[])vStatus;
        RefEntity = (object[])vRefEntity;
        FeatComp = (object[])vFeatComp;

        Debug.Print("");
        for (i = 0; i <= nRefCount - 1; i++)
        {
            Debug.Print("    Model path + name      = " + ModelPathName[i]);
            Debug.Print("    Component path + name    = " + ComponentPathName[i]);
            Debug.Print("    Feature                      = " + Feature[i]);
            Debug.Print("    Data type                   = " + DataType[i]);
            Debug.Print("    Status                      = " + System.Convert.ToString(Status[i]));
            Debug.Print("    Reference entity            = " + RefEntity[i]);
            Debug.Print("    Feature component           = " + FeatComp[i]);

            Debug.Print(" ");
        }
        Debug.Print("    Config option          = " + nConfigOpt);
        Debug.Print("    Config name           = " + sConfigName);
    }

    break;

// Part or assembly
default:
    nRefCount = swModDocExt.ListExternalFileReferencesCount();
    swModDocExt.ListExternalFileReferences2(out vModelPathName, out vComponentPathName, out vFeature, out
vDataType, out vStatus, out vRefEntity, out vFeatComp, out vConfigOpt, out vConfigName);

    Debug.Print("Model name = " + swModel.GetPathName());
    Debug.Print("    Reference count      = " + System.Convert.ToString(nRefCount));

    if (nRefCount >= 1)
    {
        object[] ModelPathName = new object[nRefCount - 1];
        object[] ComponentPathName = new object[nRefCount - 1];
        object[] Feature = new object[nRefCount - 1];
        object[] DataType = new object[nRefCount - 1];
        int[] Status = new int[nRefCount - 1];
        object[] RefEntity = new object[nRefCount - 1];
        object[] FeatComp = new object[nRefCount - 1];
        int[] ConfigOpt = new int[nRefCount - 1];
        object[] ConfigName = new object[nRefCount - 1];

        ModelPathName = (object[])vModelPathName;
        ComponentPathName = (object[])vComponentPathName;
        Feature = (object[])vFeature;
        DataType = (object[])vDataType;
        Status = (int[])vStatus;

```

```

RefEntity = (object[])vRefEntity;
FeatComp = (object[])vFeatComp;
ConfigOpt = (int[])vConfigOpt;
ConfigName = (object[])vConfigName;

```

```

Debug.Print("");
for (i = 0; i <= nRefCount - 1; i++)
{
    Debug.Print("    Model path + name    = " + ModelPathName[i]);
    Debug.Print("    Component path + name = " + ComponentPathName[i]);
    Debug.Print("    Feature                = " + Feature[i]);
    Debug.Print("    Data type              = " + DataType[i]);
    Debug.Print("    Status                 = " + System.Convert.ToString(Status[i]));
    Debug.Print("    Reference entity       = " + RefEntity[i]);
    Debug.Print("    Feature component      = " + FeatComp[i]);
    Debug.Print("    Config option          = " + ConfigOpt[i]);
    Debug.Print("    Config name            = " + ConfigName[i]);
    Debug.Print(" ");
}
}

```

```

    break;
}

```

```

}

```

```

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>

```

```

public SldWorks swApp;

```

```

    }
}

```

Get External References (VBA)

This example shows how to get a list of the external references for a selected component, selected feature, or document.

```
'-----
' Preconditions:
' 1. Open an assembly or part document.
' 2. Select:
'   * a component in an assembly document.
'   - or -
'   * a feature in an assembly or part document.
'   - or -
'   * Nothing in either type of document.
'
' Postconditions: Examine the Immediate window
' to see the name of the part or assembly document
' and the external references for the
' selected component, selected feature, or
' document.
'-----
```

Option Explicit

```
Sub main()
    Dim swApp                As SldWorks.SldWorks
    Dim swModel              As SldWorks.ModelDoc2
    Dim swModDocExt          As SldWorks.ModelDocExtension
    Dim swSelMgr             As SldWorks.SelectionMgr
    Dim swFeat              As SldWorks.Feature
    Dim swComp              As SldWorks.Component2
    Dim vModelPathName       As Variant
    Dim vComponentPathName   As Variant
    Dim vFeature             As Variant
    Dim vDataType            As Variant
    Dim vStatus              As Variant
    Dim vRefEntity           As Variant
    Dim vFeatComp            As Variant
    Dim nConfigOpt           As Long
    Dim sConfigName          As String
    Dim vConfigOpt           As Variant
    Dim vConfigName          As Variant
    Dim nRefCount            As Long
    Dim nSelType             As Long
    Dim i                   As Long

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager

    Set swModDocExt = swModel.Extension
    nSelType = swSelMgr.GetSelectedObjectType3(1, -1)

    Select Case nSelType
        ' Selected component in an assembly document
        Case swSelCOMPONENTS
            Set swComp = swSelMgr.GetSelectedObjectsComponent3(1, -1)
            nRefCount = swComp.ListExternalFileReferencesCount
            swComp.ListExternalFileReferences2 vModelPathName, vComponentPathName, vFeature, vDataType, vStatus, vRefEntity, vFeatComp, nConfigOpt,
sConfigName
            Set swModel = swComp.GetModelDoc2

            Debug.Print "Model name = " + swModel.GetPathName
            Debug.Print "   Reference count      = " + Str(nRefCount)
            Debug.Print ""
            For i = 0 To nRefCount - 1
                Debug.Print "   Model path + name      = " + vModelPathName(i)
                Debug.Print "   Component path + name = " + vComponentPathName(i)
                Debug.Print "   Feature                = " + vFeature(i)
                Debug.Print "   Data type              = " + vDataType(i)
                Debug.Print "   Status                 = " + Str(vStatus(i))
                Debug.Print "   Reference entity       = " + vRefEntity(i)
                Debug.Print "   Feature component      = " + vFeatComp(i)
            Next i
            Debug.Print "   Configuration option  = " & nConfigOpt
            Debug.Print "   Configuration name    = " & sConfigName
            Debug.Print ""

            ' Selected feature in a part or assembly document
            Case swSelBODYFEATURES, swSelSKETCHES
                Set swFeat = swSelMgr.GetSelectedObject6(1, -1)
                nRefCount = swFeat.ListExternalFileReferencesCount
                swFeat.ListExternalFileReferences2 vModelPathName, vComponentPathName, vFeature, vDataType, vStatus, vRefEntity, vFeatComp, nConfigOpt,
sConfigName

                Debug.Print "   Reference count      = " + Str(nRefCount)
                Debug.Print ""
                For i = 0 To nRefCount - 1
                    Debug.Print "   Model path + name      = " + vModelPathName(i)
                    Debug.Print "   Component path + name = " + vComponentPathName(i)
                    Debug.Print "   Feature                = " + vFeature(i)
                    Debug.Print "   Data type              = " + vDataType(i)
                    Debug.Print "   Status                 = " + Str(vStatus(i))
                    Debug.Print "   Reference entity       = " + vRefEntity(i)
                    Debug.Print "   Feature component      = " + vFeatComp(i)
                Next i
            End Select
    End Select
End Sub
```

```

Next i
    Debug.Print "    Configuration option  = " & nConfigOpt
    Debug.Print "    Configuration name   = " & sConfigName
    Debug.Print " "

    ' Part or assembly
    Case Else
        nRefCount = swModDocExt.ListExternalFileReferencesCount
        swModDocExt.ListExternalFileReferences2 vModelPathName, vComponentPathName, vFeature, vDataType, vStatus, vRefEntity, vFeatComp, vConfigOpt,
vConfigName

        Debug.Print "Model name = " + swModel.GetPathName
        Debug.Print "    Reference count      = " + Str(nRefCount)
        Debug.Print " "
        For i = 0 To nRefCount - 1
            Debug.Print "    Model path + name      = " + vModelPathName(i)
            Debug.Print "    Component path + name = " + vComponentPathName(i)
            Debug.Print "    Feature                = " + vFeature(i)
            Debug.Print "    Data type              = " + vDataType(i)
            Debug.Print "    Status                 = " + Str(vStatus(i))
            Debug.Print "    Reference entity       = " + vRefEntity(i)
            Debug.Print "    Feature component      = " + vFeatComp(i)
            Debug.Print "    Configuration option  = " & vConfigOpt(i)
            Debug.Print "    Configuration name    = " & vConfigName(i)
            Debug.Print " "
        Next i
    End Select

End Sub

```

Get External References (VB.NET)

This example shows how to get a list of the external references for a selected component, selected feature, or document.

```

'-----
' Preconditions:
' 1. Open an assembly or part document.
' 2. Select:
'   * a component in an assembly document.
'     - or -
'   * a feature in an assembly or part document.
'     - or -
'   * Nothing in either type of document.
'
' Postconditions: Examine the Immediate window
' to see the name of the part or assembly document
' and the external references for the selected
' component, selected feature, or document.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swFeat As Feature
        Dim swComp As Component2
        Dim vModelPathName As Object = Nothing
        Dim vComponentPathName As Object = Nothing
        Dim vFeature As Object = Nothing
        Dim vDataType As Object = Nothing
        Dim vStatus As Object = Nothing
        Dim vRefEntity As Object = Nothing
        Dim vFeatComp As Object = Nothing
        Dim vConfigOpt As Object = Nothing
        Dim vConfigName As Object = Nothing
        Dim nConfigOpt As Integer
        Dim sConfigName As String
        Dim nRefCount As Integer
        Dim nSelType As Integer
        Dim i As Integer

        swApp = CreateObject("SldWorks.Application")
        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager

        swModDocExt = swModel.Extension
        nSelType = swSelMgr.GetSelectedObjectType3(1, -1)

        Select Case nSelType

            ' Selected component in an assembly
            Case swSelectType_e.swSelCOMPONENTS
                swComp = swSelMgr.GetSelectedObjectsComponent3(1, -1)
                nRefCount = swComp.ListExternalFileReferencesCount
                swComp.ListExternalFileReferences2(vModelPathName, vComponentPathName, vFeature, vDataType, vStatus, vRefEntity,
                vFeatComp, nConfigOpt, sConfigName)

                swModel = swComp.GetModelDoc2

                Debug.Print("Model name = " + swModel.GetPathName)
                Debug.Print("Reference count = " + Str(nRefCount))
                Debug.Print("")
                For i = 0 To nRefCount - 1
                    Debug.Print("Model path + name = " + vModelPathName(i))
                    Debug.Print("Component path + name = " + vComponentPathName(i))
                    Debug.Print("Feature = " + vFeature(i))
                    Debug.Print("Data type = " + vDataType(i))
                    Debug.Print("Status = " + Str(vStatus(i)))
                    Debug.Print("Reference entity = " + vRefEntity(i))
                    Debug.Print("Feature component = " + vFeatComp(i))
                Next i

                Debug.Print("Configuration option = " & nConfigOpt)
                Debug.Print("Configuration name = " & sConfigName)
                Debug.Print("")

            ' Selected feature in a part or assembly
            Case swSelectType_e.swSelBODYFEATURES, swSelectType_e.swSelSKETCHES
                swFeat = swSelMgr.GetSelectedObject6(1, -1)
                nRefCount = swFeat.ListExternalFileReferencesCount
                swFeat.ListExternalFileReferences2(vModelPathName, vComponentPathName, vFeature, vDataType, vStatus, vRefEntity,

```

```

vFeatComp, nConfigOpt, sConfigName)

Debug.Print("Model name = " + swModel.GetPathName)
Debug.Print("Reference count = " + Str(nRefCount))
Debug.Print("")
For i = 0 To nRefCount - 1
    Debug.Print("Model path + name = " + vModelPathName(i))
    Debug.Print("Component path + name = " + vComponentPathName(i))
    Debug.Print("Feature = " + vFeature(i))
    Debug.Print("Data type = " + vDataType(i))
    Debug.Print("Status = " + Str(vStatus(i)))
    Debug.Print("Reference entity = " + vRefEntity(i))
    Debug.Print("Feature component = " + vFeatComp(i))
Next i

Debug.Print("Configuration option = " & nConfigOpt)
Debug.Print("Configuration name = " & sConfigName)
Debug.Print("")
' Part or assembly
Case Else
    nRefCount = swModDocExt.ListExternalFileReferencesCount
    swModDocExt.ListExternalFileReferences2(vModelPathName, vComponentPathName, vFeature, vDataType, vStatus,
vRefEntity, vFeatComp, vConfigOpt, vConfigName)
Debug.Print("Model name = " + swModel.GetPathName)
Debug.Print("Reference count = " + Str(nRefCount))
Debug.Print("")
For i = 0 To nRefCount - 1
    Debug.Print("Model path + name = " + vModelPathName(i))
    Debug.Print("Component path + name = " + vComponentPathName(i))
    Debug.Print("Feature = " + vFeature(i))
    Debug.Print("Data type = " + vDataType(i))
    Debug.Print("Status = " + Str(vStatus(i)))
    Debug.Print("Reference entity = " + vRefEntity(i))
    Debug.Print("Feature component = " + vFeatComp(i))
    Debug.Print("Configuration option = " & vConfigOpt(i))
    Debug.Print("Configuration name = " & vConfigName(i))
    Debug.Print("")
Next i
End Select

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Get Face Hatch Data Example (C#)

This example shows how to get the number of face hatches in a standard or derived (detail, section, projected, broken, etc.) drawing view and their data.

```
//-----
// Preconditions:
// 1. Open public_documents\introsw\bolt-assembly.slddrw.
// 2. Select Section View A-A in the FeatureManager design tree.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Gets face hatch data.
// 2. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace UseMaterialHatchFaceHatchCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            View swView = default(View);
            object[] vFaceHatch = null;
            FaceHatch swFaceHatch = default(FaceHatch);
            int i = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swView = (View)swSelMgr.GetSelectedObject6(1, -1);
            Debug.Print("View = " + swView.Name);
            Debug.Print(" Type = " + swView.Type);
            vFaceHatch = (object[])swView.GetFaceHatches();
            if ((vFaceHatch == null))
            {
                Debug.Print(" No face hatches in selected view.");
                return;
            }
            Debug.Print(" Number of face hatches in this view = " + (vFaceHatch.Length));
            if ((vFaceHatch != null))
            {
                Debug.Print(" Face hatches =");
                Debug.Print("");
                for (i = 0; i <= vFaceHatch.Length - 1; i++)
                {
                    swFaceHatch = (FaceHatch)vFaceHatch[i];
                    // Get face hatch data
                    // 1 radian = 180°/p = 57.295779513° or approximately 57.3°
                    Debug.Print(" Angle = " + swFaceHatch.Angle * 57.3 + " degrees");
                    Debug.Print(" Color = " + swFaceHatch.Color);
                    Debug.Print(" Definition = " + swFaceHatch.Definition);
                    Debug.Print(" Layer = " + swFaceHatch.Layer);
                    Debug.Print(" Pattern = " + swFaceHatch.Pattern);
                    Debug.Print(" Scale = " + swFaceHatch.Scale2);
                    Debug.Print(" SolidFill = " + swFaceHatch.SolidFill);
                    Debug.Print(" Material crosshatch = " + swFaceHatch.UseMaterialHatch);
                    Debug.Print(" -----");
                }
            }
        }
    }
}
/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
```

Get Face Hatch Data Example (VBA)

This example shows how to get the number of face hatches in a standard or derived (detail, section, projected, broken, etc.) drawing view and data.

```

'-----
' Preconditions:
' 1. Open public_documents\introsw\bolt-assembly.slddrw.
' 2. Select Section View A-A in the FeatureManager design tree.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets face hatch data.
' 2. Examine the Immediate window.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swView As SldWorks.View
    Dim vFaceHatch As Variant
    Dim swFaceHatch As SldWorks.FaceHatch
    Dim i As Long

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swView = swSelMgr.GetSelectedObject6(1, -1)
    Debug.Print "View   = " & swView.Name
    Debug.Print " Type = " & swView.Type
    vFaceHatch = swView.GetFaceHatches
    If IsEmpty(vFaceHatch) Then
        Debug.Print " No face hatches in selected view."
        Exit Sub
    End If
    Debug.Print " Number of face hatches in this view = " & (UBound(vFaceHatch) + 1)
    If Not IsEmpty(vFaceHatch) Then
        Debug.Print " Face hatches ="
        Debug.Print ""
        For i = 0 To UBound(vFaceHatch)
            Set swFaceHatch = vFaceHatch(i)
            ' Get face hatch data
            ' 1 radian = 180°/p = 57.295779513° or approximately 57.3°
            Debug.Print " Angle           = " & swFaceHatch.Angle * 57.3 & " degrees"
            Debug.Print " Color           = " & swFaceHatch.Color
            Debug.Print " Definition      = " & swFaceHatch.Definition
            Debug.Print " Layer          = " & swFaceHatch.Layer
            Debug.Print " Pattern        = " & swFaceHatch.Pattern
            Debug.Print " Scale          = " & swFaceHatch.Scale2
            Debug.Print " SolidFill      = " & swFaceHatch.SolidFill
            Debug.Print " Material crosshatch = " & swFaceHatch.UseMaterialHatch
            Debug.Print " -----"
        Next i
    End If
End Sub

```

Get Face Hatch Data Example (VB.NET)

This example shows how to get the number of face hatches in a standard or derived (detail, section, projected, broken, etc.) drawing view and data.

```

'-----
' Preconditions:
' 1. Open public_documents\introsw\bolt-assembly.slddrw.
' 2. Select Section View A-A in the FeatureManager design tree.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets face hatch data.
' 2. Examine the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public
    Sub main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swView As View
        Dim vFaceHatch As Object
        Dim swFaceHatch As FaceHatch
        Dim i As Long

        swApp = CreateObject("SldWorks.Application")
        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swView = swSelMgr.GetSelectedObject6(1,
-1)
        Debug.Print("View    =
" & swView.Name)
        Debug.Print("    Type =
" & swView.Type)

        vFaceHatch = swView.GetFaceHatches
        If IsNothing(vFaceHatch) Then
            Debug.Print(" No face hatches in selected view.")
            Exit Sub
        End If
        Debug.Print(" Number of face hatches in this view = " & (UBound(vFaceHatch) + 1))
        If Not IsNothing(vFaceHatch) Then
            Debug.Print(" Face hatches =")
            Debug.Print("")
            For i = 0 To UBound(vFaceHatch)
                swFaceHatch = vFaceHatch(i)
                ' Get face hatch data
                ' 1 radian = 180°/p = 57.295779513° or approximately 57.3°
                Debug.Print("    Angle    = " & swFaceHatch.Angle * 57.3 & " degrees")
                Debug.Print("    Color    = " & swFaceHatch.Color)
                Debug.Print("    Definition = " & swFaceHatch.Definition)
                Debug.Print("    Layer    = " & swFaceHatch.Layer)
                Debug.Print("    Pattern   = " & swFaceHatch.Pattern)
                Debug.Print("    Scale    = " & swFaceHatch.Scale2)
                Debug.Print("    SolidFill = " & swFaceHatch.SolidFill)
                Debug.Print("    Material crosshatch = " & swFaceHatch.UseMaterialHatch)
                Debug.Print("    -----")
            Next i
        End If

        End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Get Faces Adjacent to Vertex Example (VBA)

This example shows how to get the faces adjacent to the selected vertex.

```
'-----  
' Preconditions: Open a part document and select a vertex.  
'  
' Postconditions:  
' 1. Selects the faces adjacent to the selected vertex.  
' 2. Examine the graphics area.  
'-----  
Option Explicit
```

```
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swVert As SldWorks.Vertex  
    Dim vFaceArr As Variant  
    Dim vFace As Variant  
    Dim swFace As SldWorks.Face2  
    Dim swEnt As SldWorks.Entity  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swVert = swSelMgr.GetSelectedObject6(1, -1)  
  
    swModel.ClearSelection2 True  
  
    vFaceArr = swVert.GetAdjacentFaces  
    For Each vFace In vFaceArr  
        Set swFace = vFace  
        Set swEnt = swFace  
        bRet = swEnt.Select4(True, Nothing)  
    Next vFace  
  
End Sub
```

Get Faces Affected by Draft Feature Example (C#)

This example shows how to get the faces affected by a draft feature.

NOTE: In SOLIDWORKS Design, a feature is comprised of faces. The faces created by a feature can be retrieved using `IFeature::GetFaces`. However, certain types of features only affect existing faces and do not create any new faces. Examples of these types of features are draft and scale features. For these features, `IFeature::GetFaces` returns an empty array (no faces). This sample code shows how to use `IDraftFeatureData2::GetDraftedEntities` to get the faces affected by a draft feature.

```
//-----
// Preconditions:
// 1. Open a part or assembly with a draft feature.
// 2. If you opened an assembly, then fully resolve the assembly if necessary.
// 3. Select a draft feature.
//
// Postconditions:
// 1. Deselects the draft feature.
// 2. Selects the faces affected by the draft feature.
// 3. Gets the type of draft feature as defined in swDraftType_e.
// 4. Examine the Immediate window and graphics area.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace GetFacesForDraftFeature_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            SelectData swSelData = default(SelectData);
            Feature swFeat = default(Feature);
            DraftFeatureData2 swDraftFeat = default(DraftFeatureData2);
            object vEnt = null;
            object[] vEntArr = null;
            Entity swEnt = default(Entity);
            bool bRet = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swSelData = (SelectData)swSelMgr.CreateSelectData();
            swFeat = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            swDraftFeat = (DraftFeatureData2)swFeat.GetDefinition();

            // Type as defined in swDraftType_e
            Debug.Print(swDraftFeat.Type.ToString());

            swModel.ClearSelection2(true);

            // No need to roll back to get affected faces
            vEntArr = (object[])swDraftFeat.GetDraftedEntities();
            foreach (object vEnt_loopVariable in vEntArr)
            {
                vEnt = vEnt_loopVariable;
                swEnt = (Entity)vEnt;
                bRet = swEnt.Select4(true, swSelData);
            }

            public SldWorks swApp;
        }
    }
}
```